Self-supervised Learning
Augmentations & Pretext Tasks

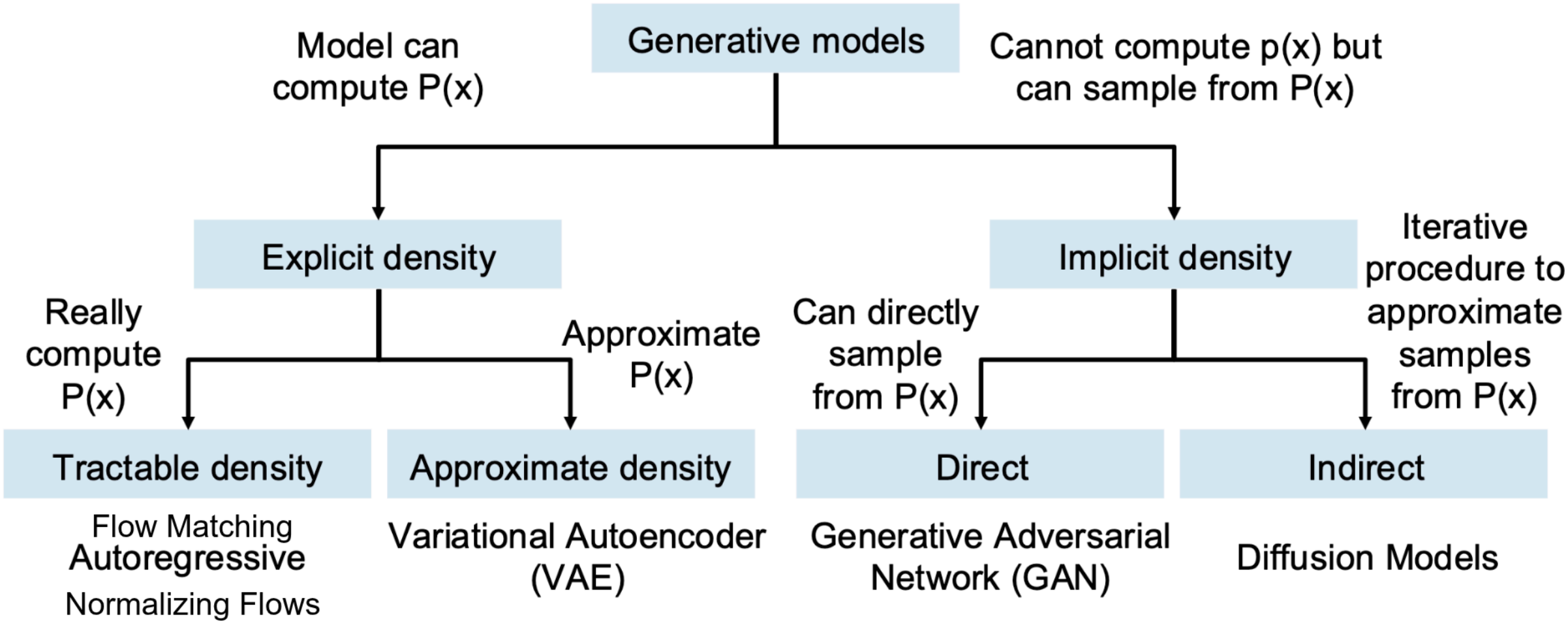
Hermann Blum

UNIVERSITÄT BONN

LAMARR INSTITUTE FOR MACHINE LEARNING AND ARTIFICIAL INTELLIGENCE

Robot Perception and Learning Lab
rpl.uni-bonn.de

Previous Lectures: Generative Models



Previous Lectures: Generative Models

General Idea: To generate samples in X, models need to learn some underlying structure of X.

Feature Extraction

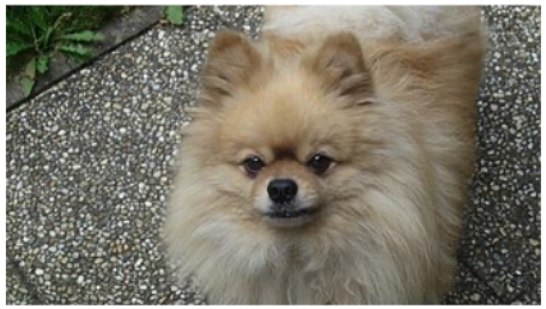
Goal: Map from X into a space Z that can be used as an ‚abstraction' of the data that is more convenient to process. E.g., points in Z often have a meaningful distance metric.

Feature Extraction

Goal: Map from X into a space Z that can be used as an ‚abstraction' of the data that is more convenient to process. E.g., points in Z often have a meaningful distance metric.

$\{a_1, a_2, a_3, \dots, a_N\}$

$\{b_1, b_2, b_3, \dots, b_N\}$



$\{c_1, c_2, c_3, \dots, c_N\}$

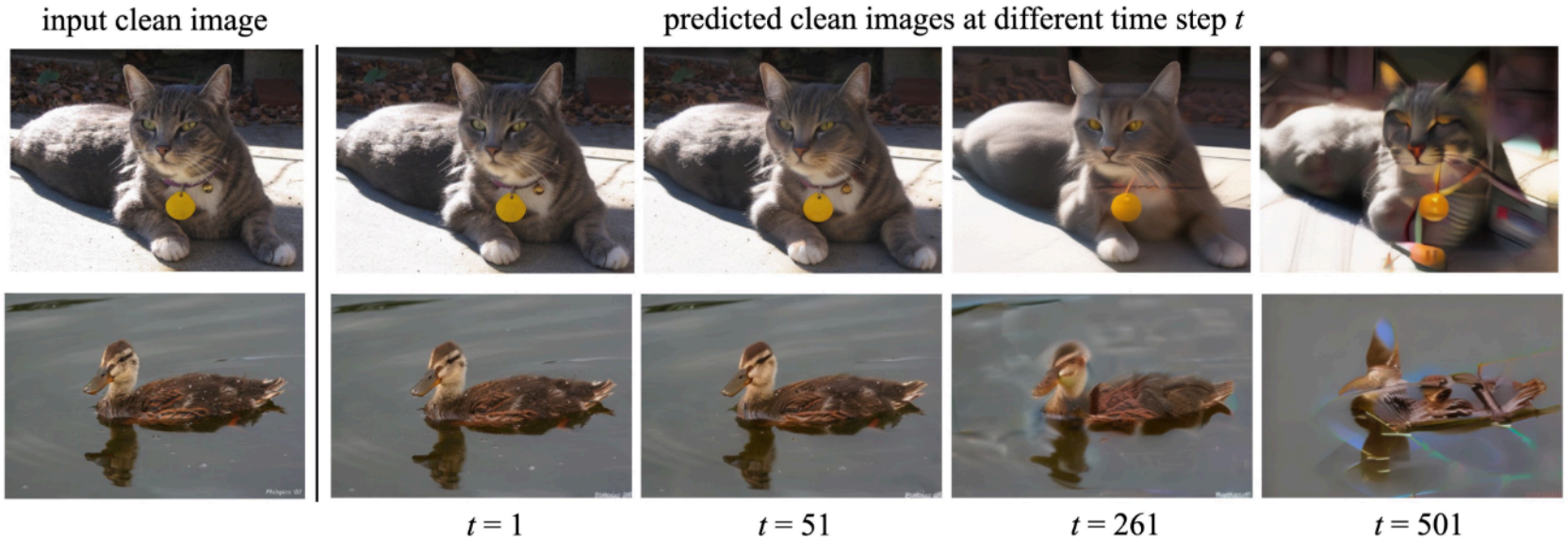
Euclidean Distance $d = \sqrt{(a-b)^2}$

Cosine Distance $d = \frac{a \cdot b}{|a| \cdot |b|}$

Features from Generative Models

General Idea: To generate samples in X, models need to learn some underlying structure of X.

Example: Diffusion Features -> noise level correlates with semantic abstraction



Features from Generative Models

General Idea: To generate samples in X, models need to learn some underlying structure of X.

Example:
Interpolations in latent space

The next few lectures: Representation Learning

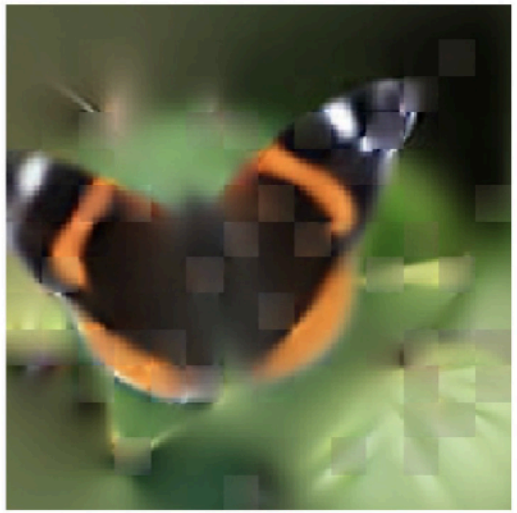
General Idea: Learn invariances or co-variances in the data to keep only relevant information in the representation.

The next few lectures: Representation Learning

General Idea: Learn invariances or co-variances in the data to keep only relevant information in the representation.

Example: Masked Autoencoder -> encode overall image content, invariant to masking

Masked input: 80%



MAE's guess

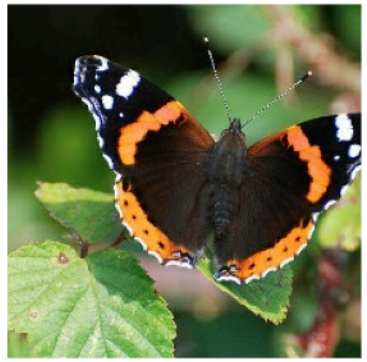
Augmentations

Masking

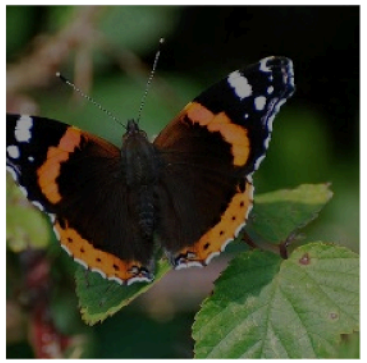
Augmentations

Masking

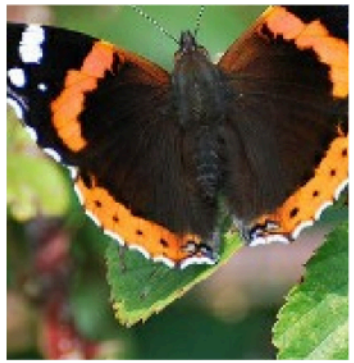
Rotation



Flipping



Brightness

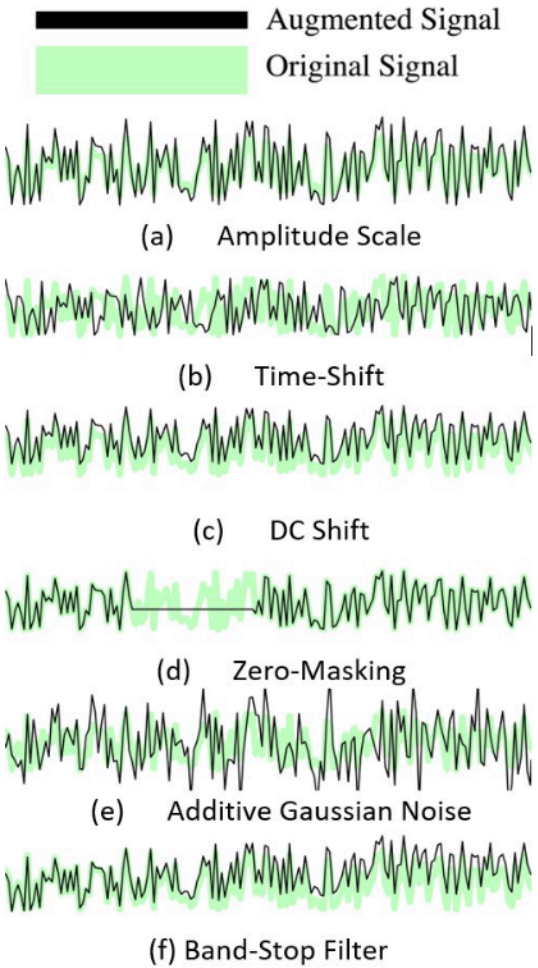


Crop

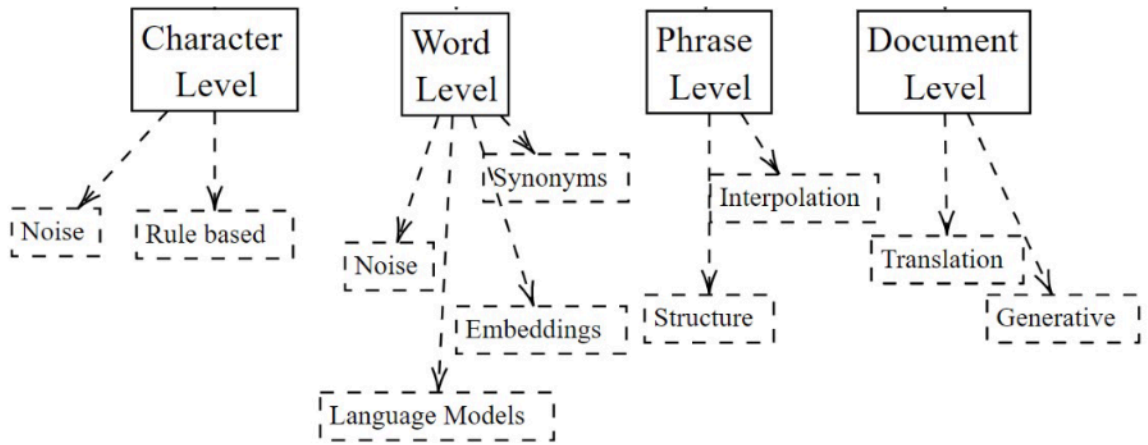
Color

Noise
(here: Gaussian Blur)

Augmentations for other Modalities

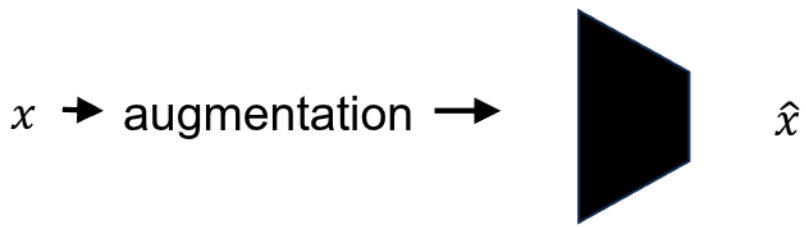


Graphic by MHR Group



Graphic by Bayer et al, 2021

SSL with Reconstruction



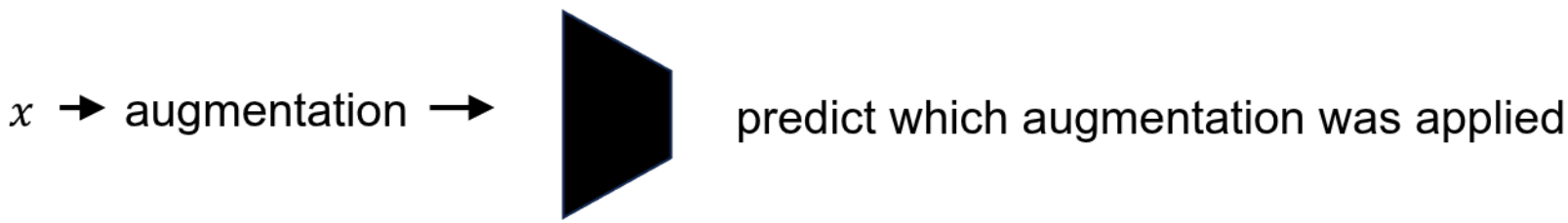
reconstruct the input, i.e. be invariant about the augmentation

Mask Inpainting

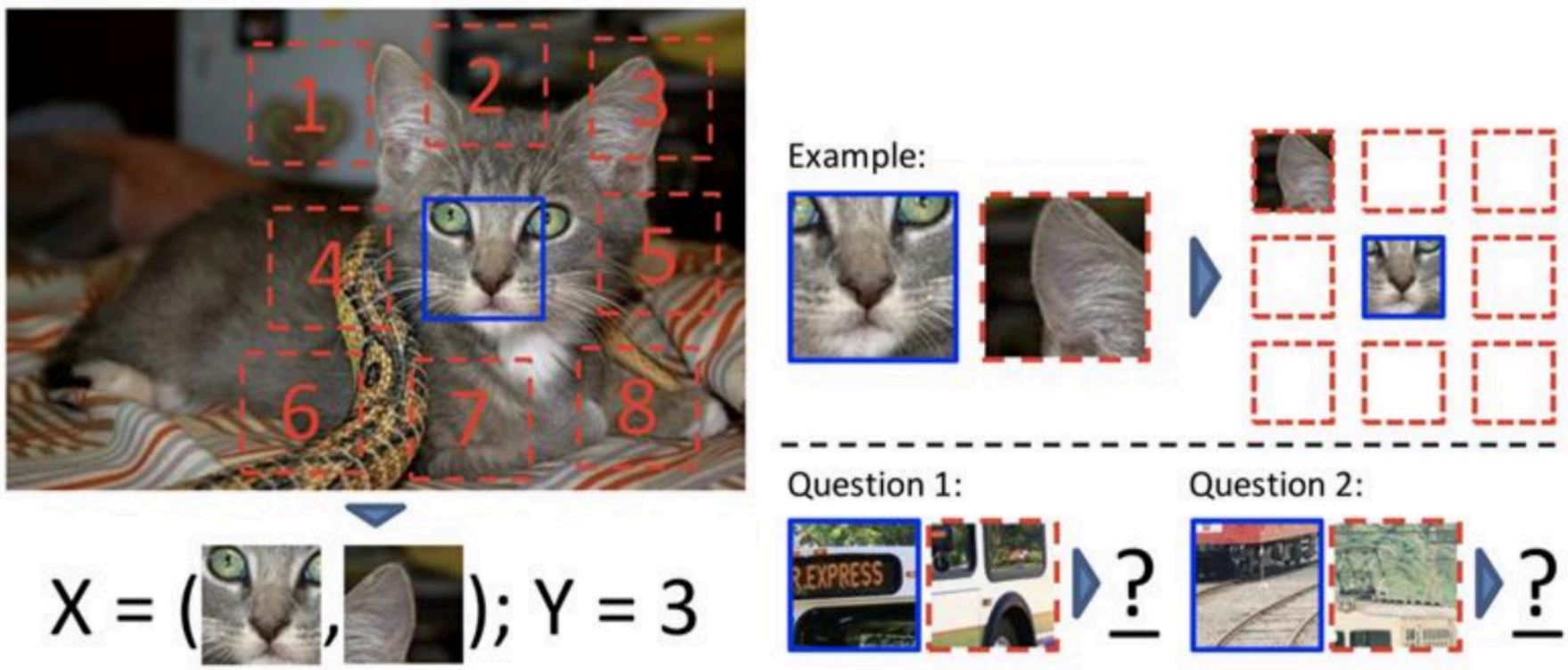
Context Encoders: Feature Learning by Inpainting (Pathak et al., 2016)

Source: Pathak et al., 2016

SSL with Pretext Tasks



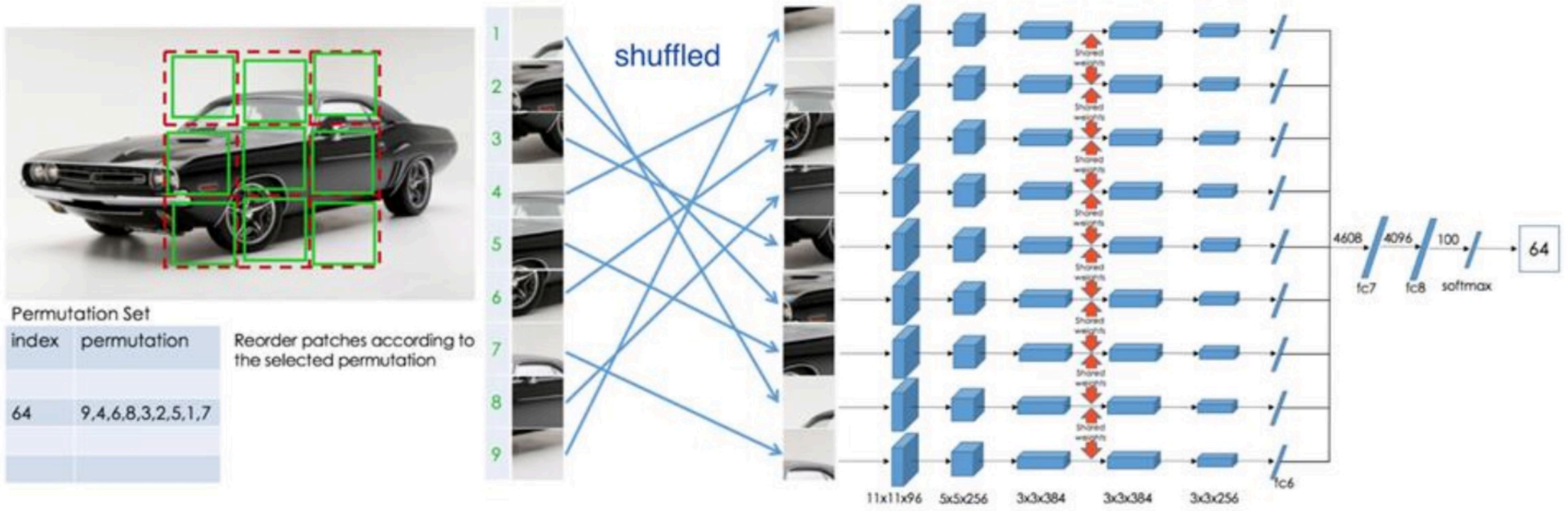
predict relative patch location



(Image source: Doersch et al., 2015)

slide by Li et al, Stanford CS231n

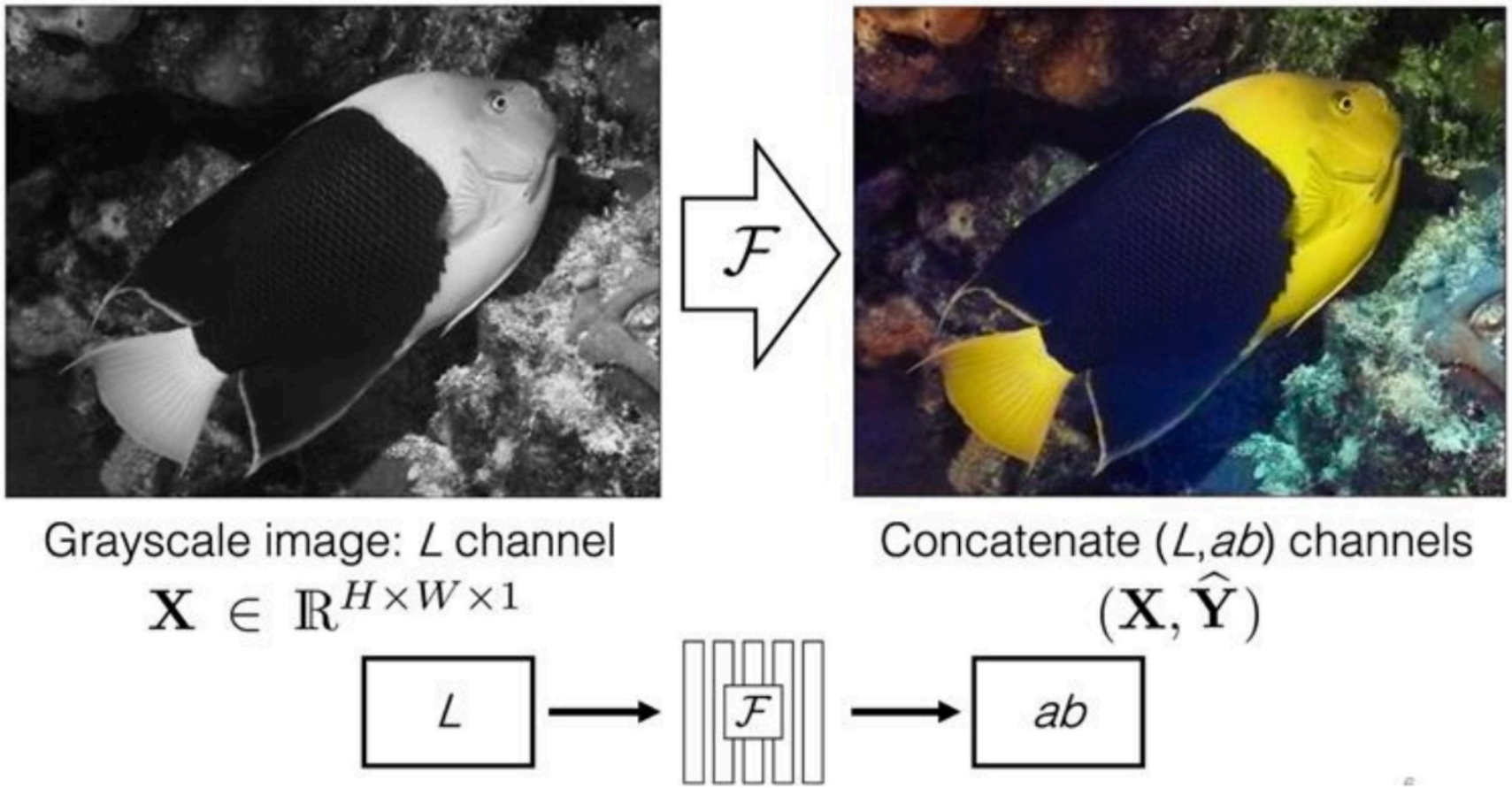
solving jigsaw puzzles



(Image source: Noroozi & Favaro, 2016)

slide by Li et al, Stanford CS231n

Image Re-coloring



Source: Richard Zhang / Phillip Isola

Video Coloring

Idea: model the temporal coherence of colors in videos

reference frame

how should I color these frames?

t = 0

t = 1

t = 2

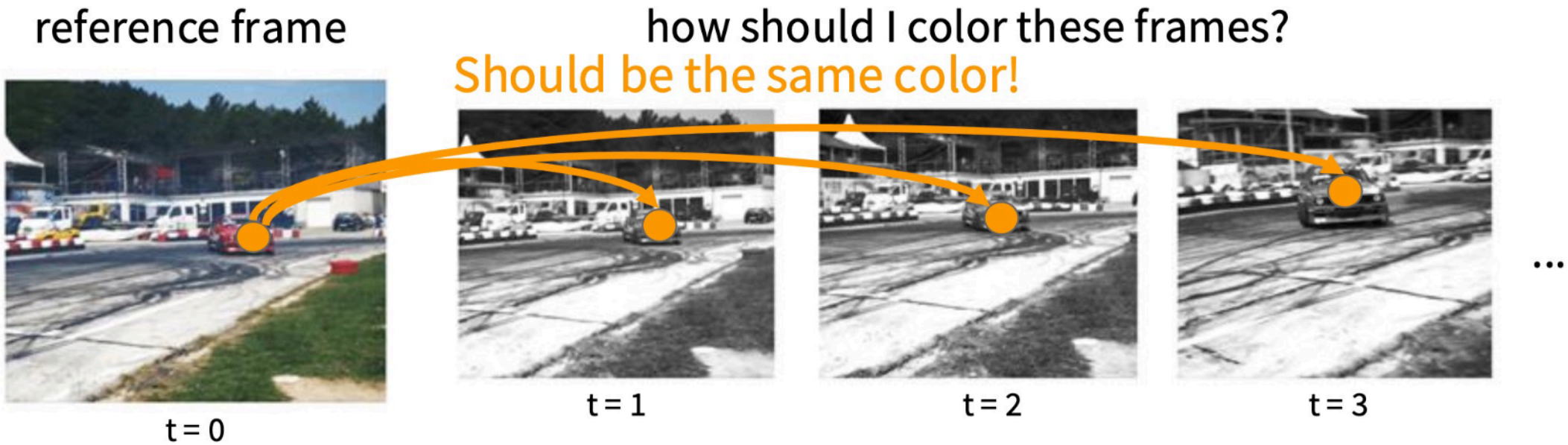
t = 3

...

Source: Vondrick et al., 2018

Video Coloring

Idea: model the temporal coherence of colors in videos

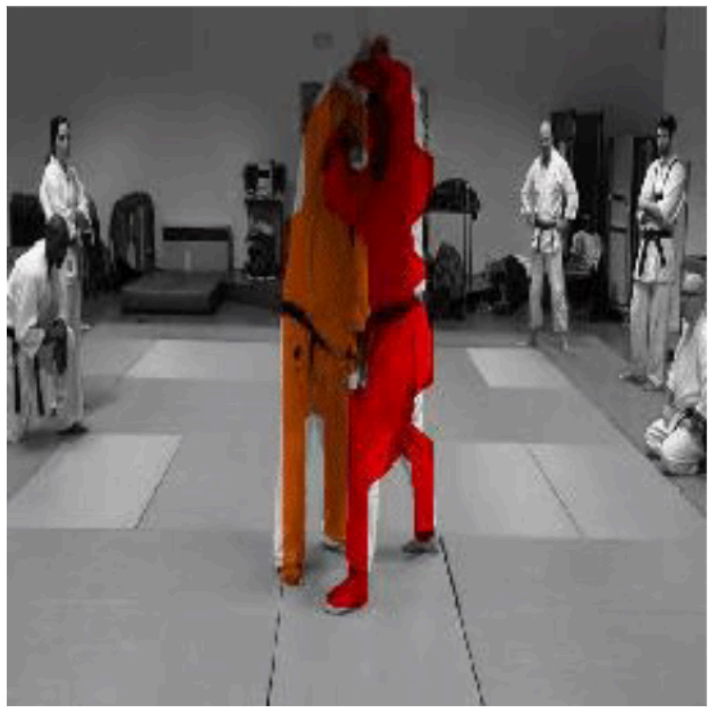


Hypothesis: learning to color video frames should allow model to learn to track regions or objects without labels!

Source: Vondrick et al., 2018 8

Tracking emerges from colorization

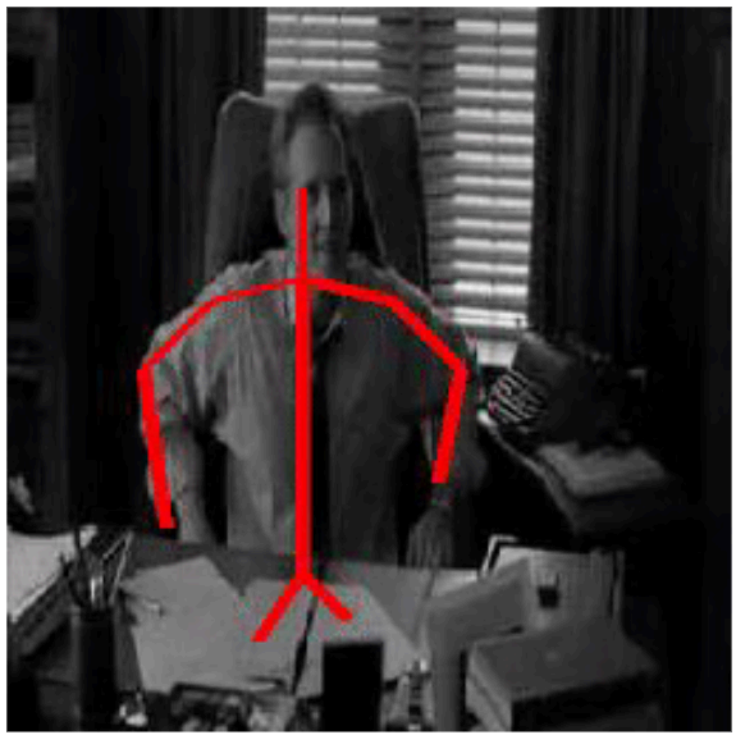
Propagating segmentation masks using learned attention.



source: https://research.google/blog/self-supervised-tracking-via-video-colorization/

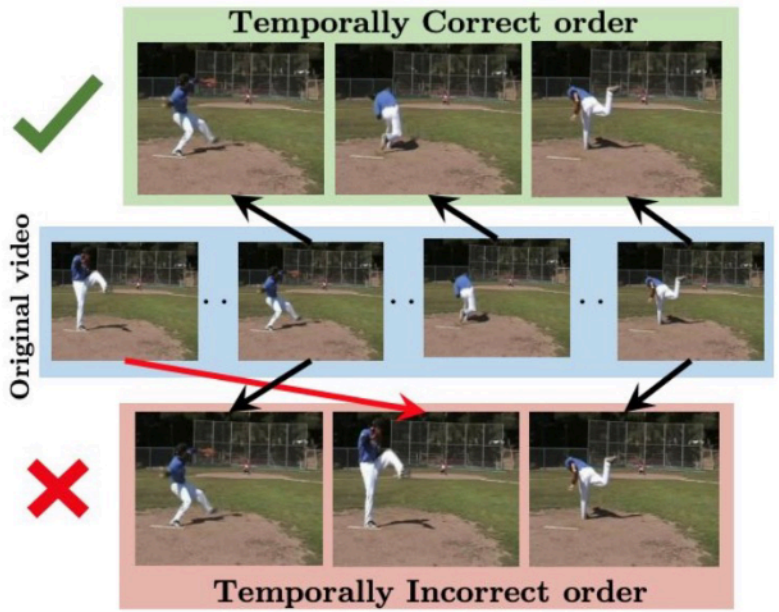
Tracking emerges from colorization

Propagating skeleton keypoints using learned attention.

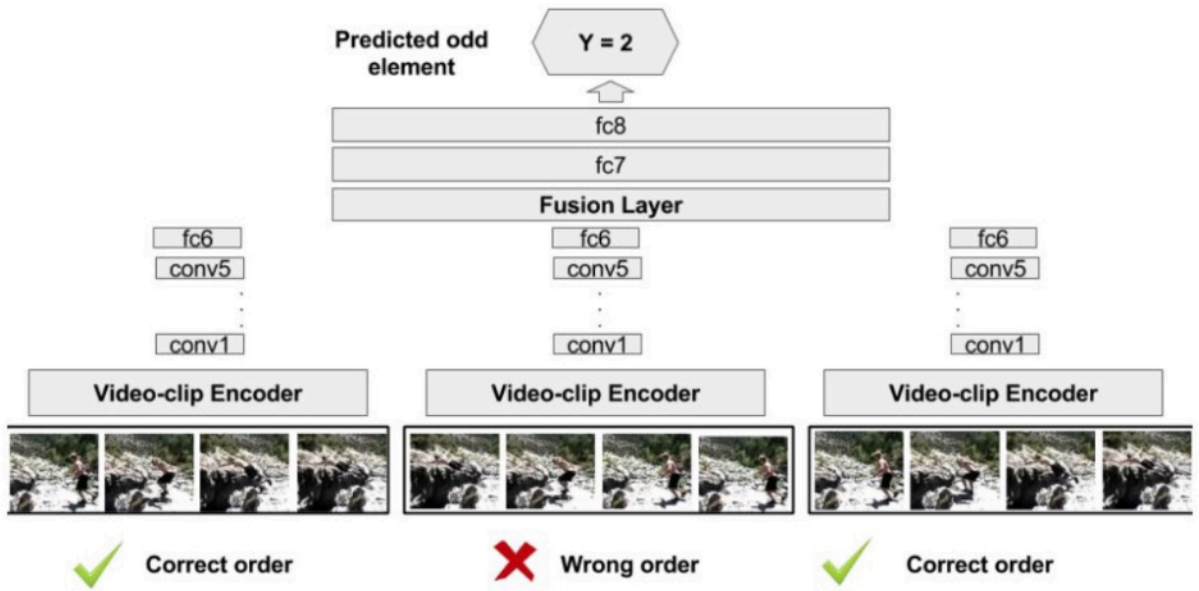


source: https://research.google/blog/self-supervised-tracking-via-video-colorization/

Temporal Order Augmentation



(Misra et al. 2016)



(Fernando et al. 2017)

RotNet: Representations from Rotation Prediction

Published as a conference paper at ICLR 2018

UNSUPERVISED REPRESENTATION LEARNING BY PREDICTING IMAGE ROTATIONS

Spyros Gidaris, Praveer Singh, Nikos Komodakis
University Paris-Est, LIGM
Ecole des Ponts ParisTech
{spyros.gidaris,praveer.singh,nikos.komodakis}@enpc.fr

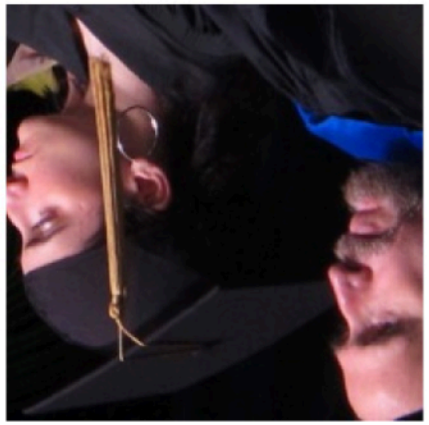
ABSTRACT

Over the last years, deep convolutional neural networks (ConvNets) have transformed the field of computer vision thanks to their unparalleled capacity to learn high level semantic image features. However, in order to successfully learn those features, they usually require massive amounts of manually labeled data, which is both expensive and impractical to scale. Therefore, unsupervised semantic feature learning, i.e., learning without requiring manual annotation effort, is of crucial importance in order to successfully harvest the vast amount of visual data that are

RotNet: Representations from Rotation Prediction

90° rotation

270° rotation


180° rotation

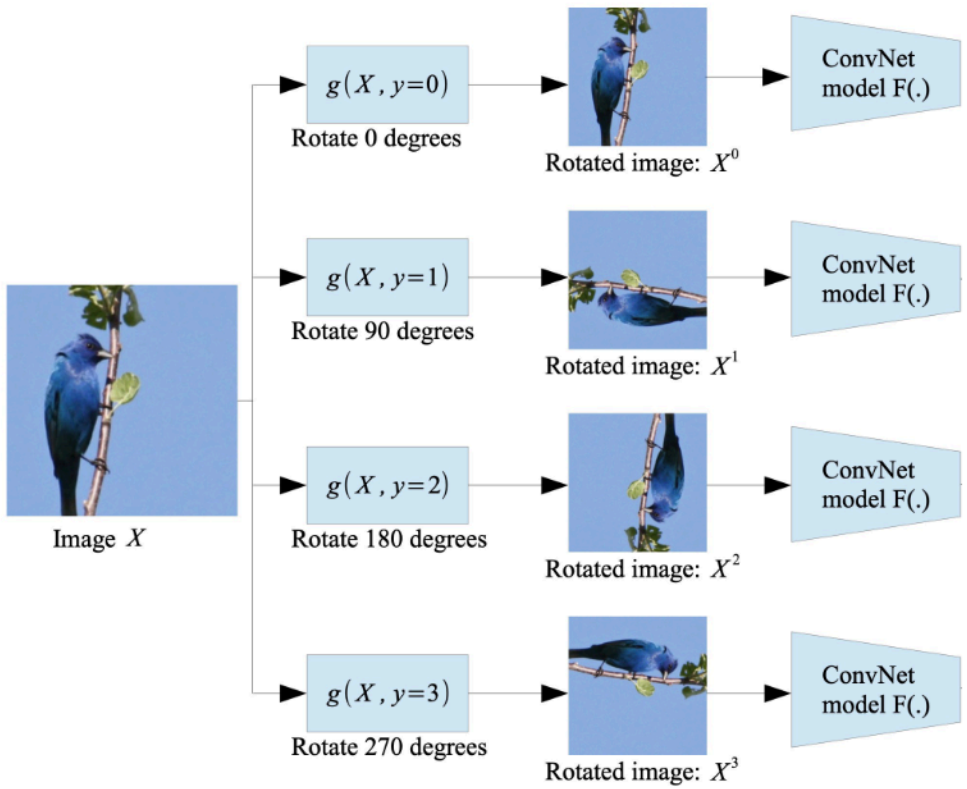
0° rotation

270° rotation

Hypothesis: a model could recognize the correct rotation of an object only if it has the “visual commonsense” of what the object should look like unperturbed.

slide adapted from Li et al, Stanford CS231n

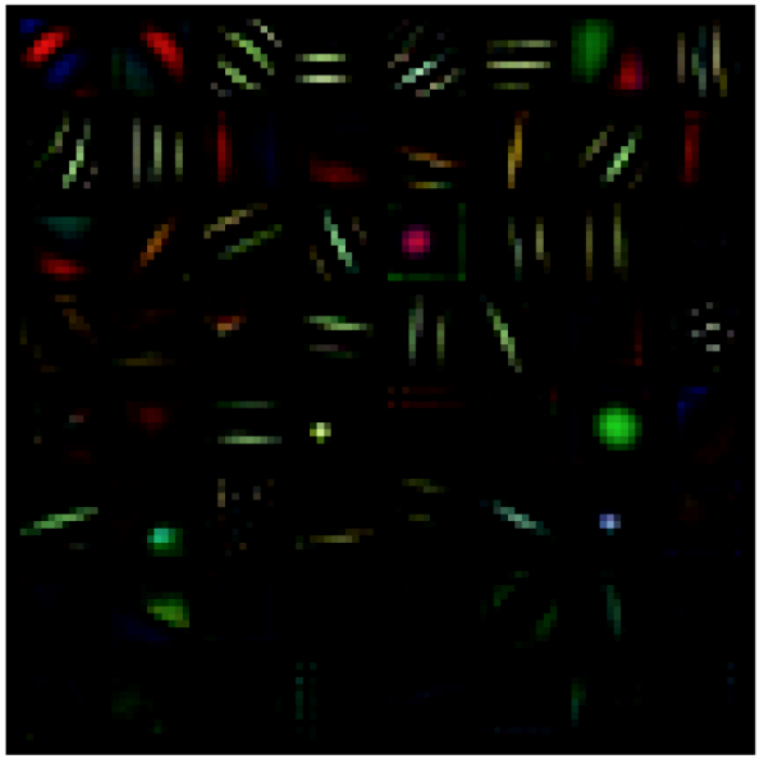
RotNet: Representations from Rotation Prediction



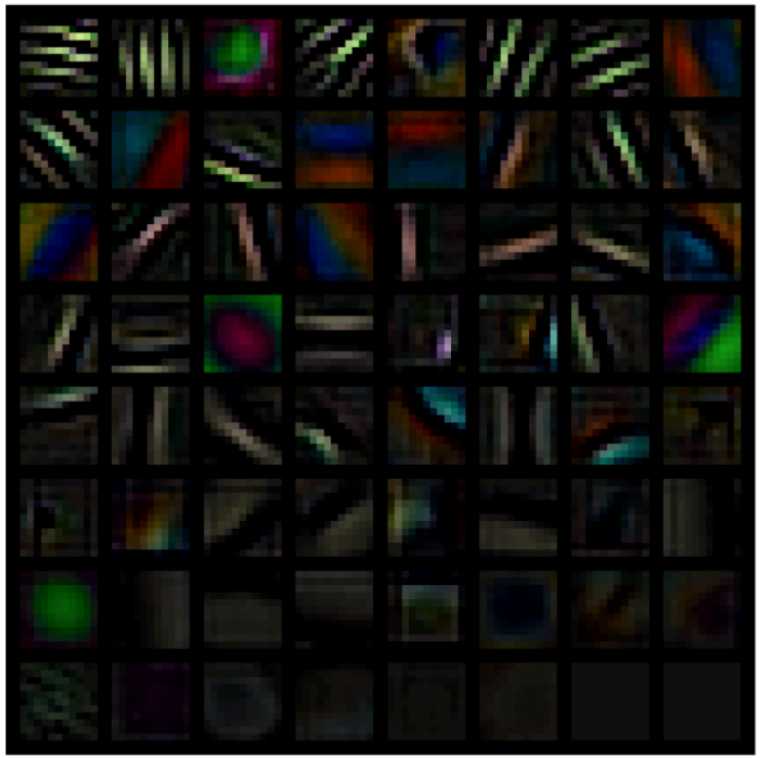
From 1 data sample, generate 4 different inputs.

The model has to predict each rotation (classification from 4 options).

The model learns similar features as for supervised classification.



(a) **Supervised**



(b) **Self-supervised to recognize rotations**

visualized filter in the first layer of AlexNet

Good features should generalize across specific tasks.

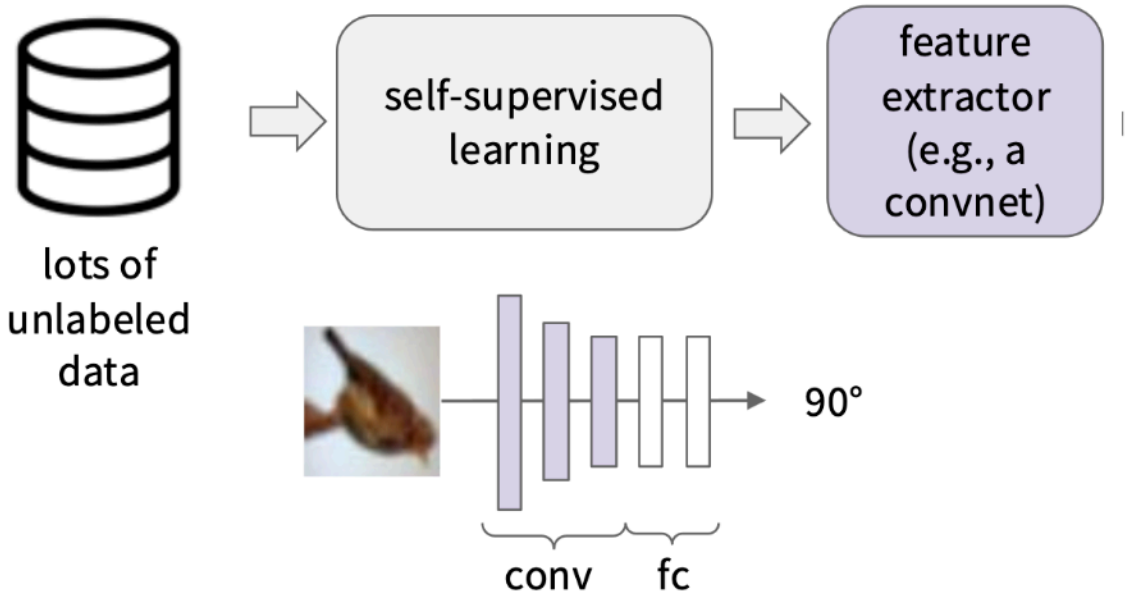
different tasks

	Classification (%mAP)		Detection (%mAP)	Segmentation (%mIoU)	
Trained layers	fc6-8	all	all	all	
ImageNet labels	78.9	79.9	56.8	48.0	pretrain on annotated images
Random		53.3	43.4	19.8	no pretraining
Random rescaled Krähenbühl et al. (2015)	39.2	56.6	45.6	32.6	
Egomotion (Agrawal et al., 2015)	31.0	54.2	43.9		other SSL approaches
Context Encoders (Pathak et al., 2016b)	34.6	56.5	44.5	29.7	
Tracking (Wang & Gupta, 2015)	55.6	63.1	47.4		
Context (Doersch et al., 2015)	55.1	65.3	51.1		
Colorization (Zhang et al., 2016a)	61.5	65.6	46.9	35.6	
BIGAN (Donahue et al., 2016)	52.3	60.1	46.9	34.9	
Jigsaw Puzzles (Noroozi & Favaro, 2016)	-	67.6	53.2	37.6	
NAT (Bojanowski & Joulin, 2017)	56.7	65.3	49.4		
Split-Brain (Zhang et al., 2016b)	63.0	67.1	46.7	36.0	
ColorProxy (Larsson et al., 2017)		65.9		38.4	
Counting (Noroozi et al., 2017)	-	67.7	51.4	36.6	
(Ours) RotNet	**70.87**	**72.97**	**54.4**	**39.1**	SSL with Rotations

How can we evaluate the quality of learned features?

- direct inspection
  - qualitatively through visualizations
  - To which changes in the input is the feature invariant?
  - With which changes in the input does the feature correlate?
- downstream applications
  - Can I directly use the feature to solve a task?
  - Is the feature useful as input to a (supervised) model for a defined task?
  - Are the SSL network parameters useful as pretraining for a supervised task?

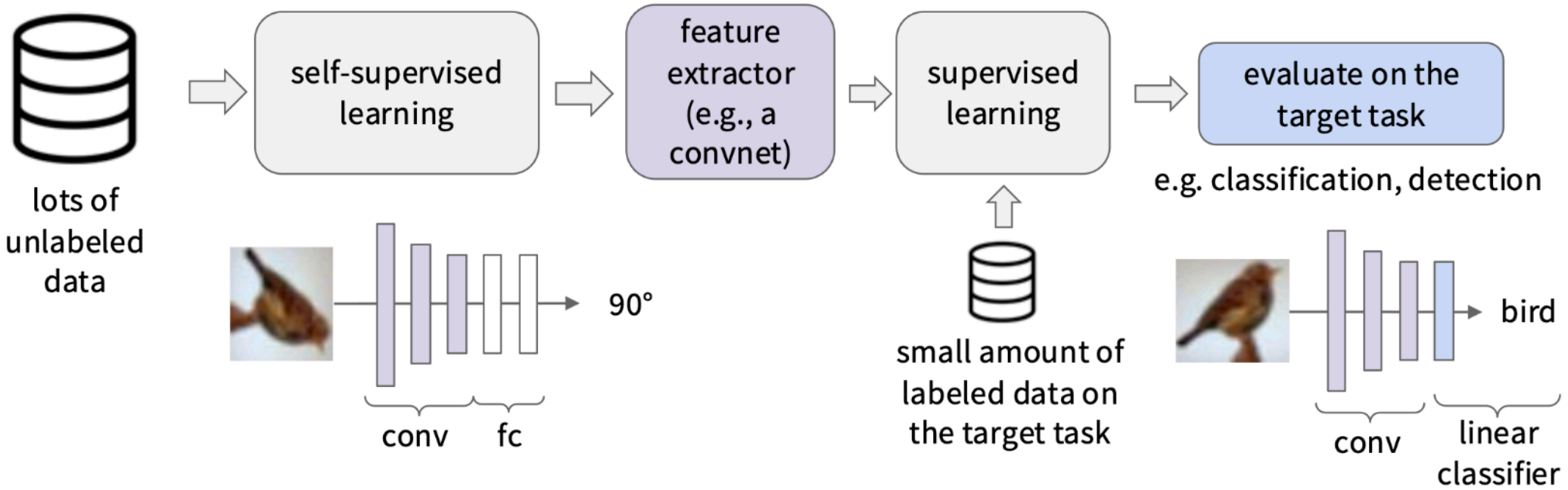
How can we evaluate the quality of learned features?



1. Learn good feature extractors from self-supervised pretext tasks, e.g., predicting image rotations

slide by Li et al, Stanford CS231n

How can we evaluate the quality of learned features?



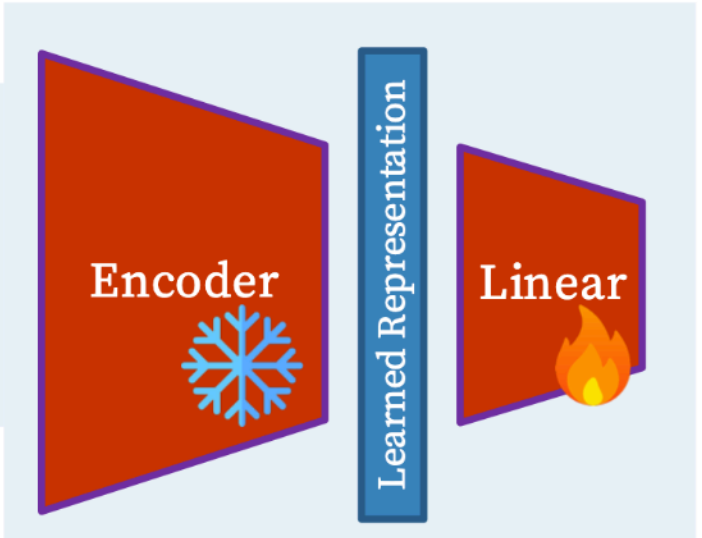
1. Learn good feature extractors from self-supervised pretext tasks, e.g., predicting image rotations

2. Attach a shallow network on the feature extractor; train the shallow network on the target task with small amount of labeled data

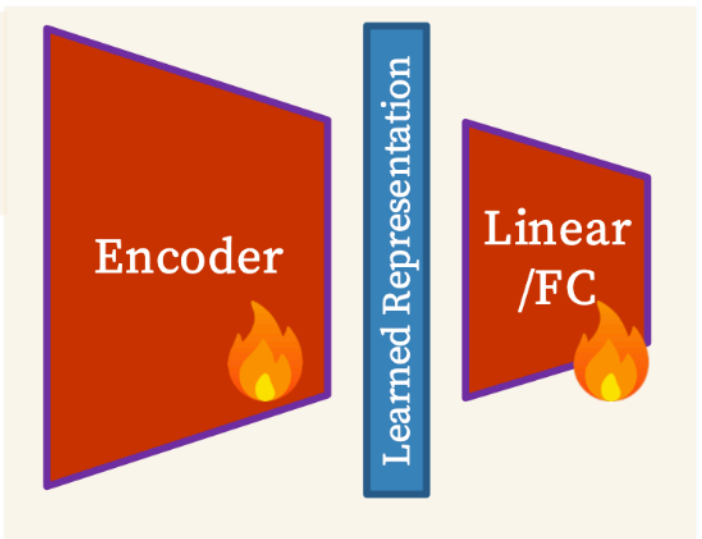
slide by Li et al, Stanford CS231n

Linear Probing vs Full Fine-tuning

- In linear probing, the pre-trained model is fixed, and only one linear layer is added at the end, to predict the labels (or produce the output). This method is used to assess the quality of representations from a pre-trained feature extraction model.



- In fine-tuning, pre-trained model is further trained (not fixed), and one or more layers, possibly with non-linearities are added.

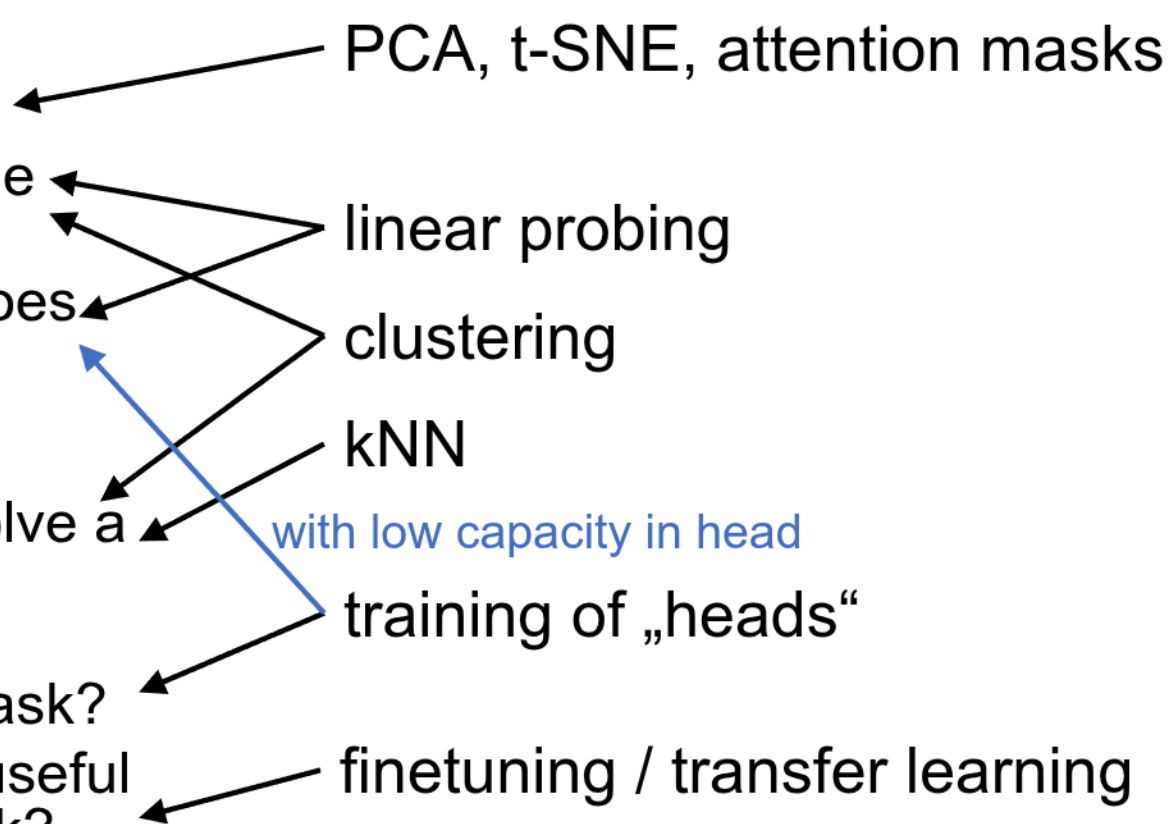


- linear probing: provides a measure of representation quality of a pre-training in restricted conditions
- fine-tuning: exploits models near-true potential to adopt for new tasks

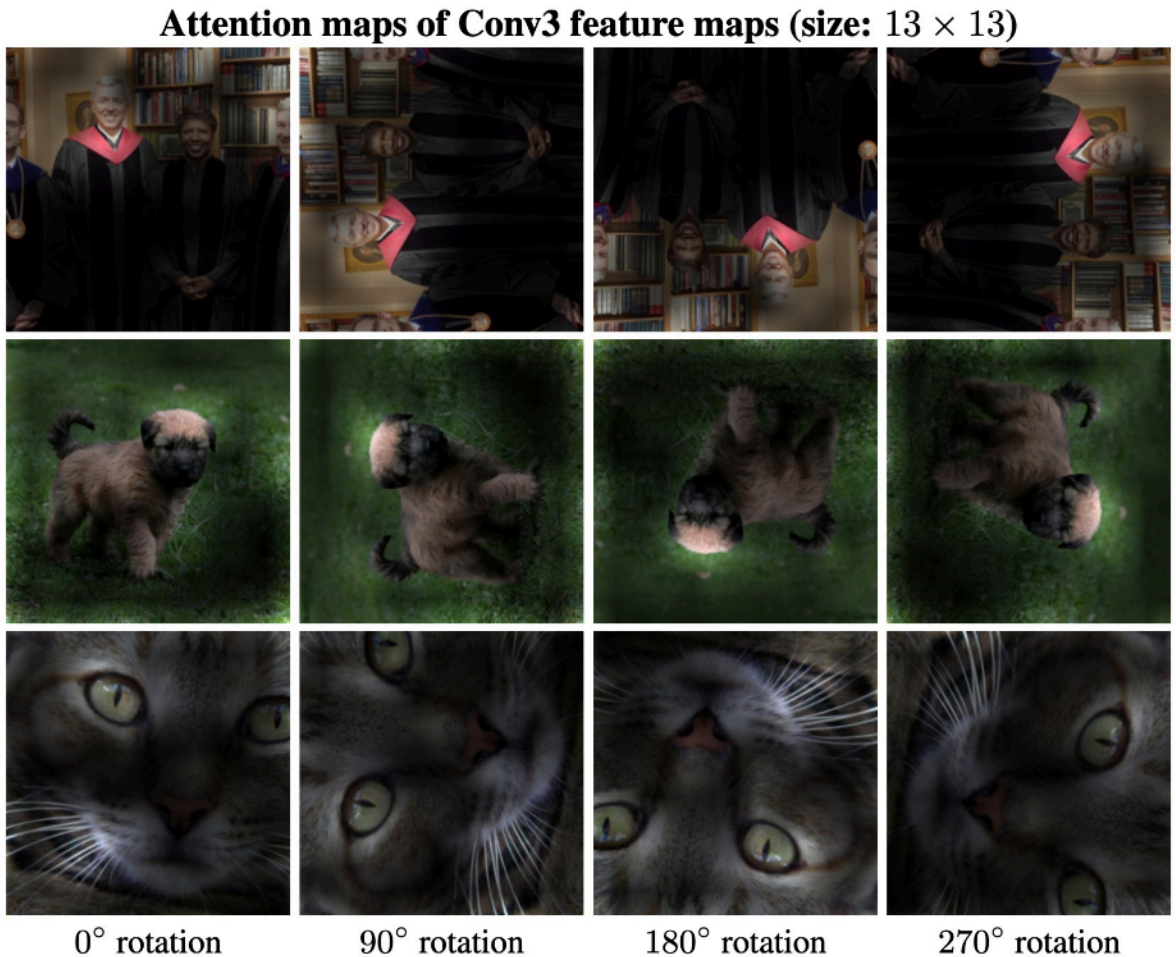
slide by Li et al, Stanford CS231n

How can we evaluate the quality of learned features?

- direct inspection
  - qualitatively through visualizations
  - To which changes in the input is the feature invariant?
  - With which changes in the input does the feature correlate?
- downstream applications
  - Can I directly use the feature to solve a task?
  - Is the feature useful as input to a (supervised) model for a defined task?
  - Are the SSL network parameters useful as pretraining for a supervised task?



What kind of evaluation is this?



What kind of evaluation is this?

different tasks

	Classification (%mAP)		Detection (%mAP)	Segmentation (%mIoU)
Trained layers	fc6-8	all	all	all
ImageNet labels	78.9	79.9	56.8	48.0
Random		53.3	43.4	19.8
Random rescaled Krähenbühl et al. (2015)	39.2	56.6	45.6	32.6
Egomotion (Agrawal et al., 2015)	31.0	54.2	43.9	
Context Encoders (Pathak et al., 2016b)	34.6	56.5	44.5	29.7
Tracking (Wang & Gupta, 2015)	55.6	63.1	47.4	
Context (Doersch et al., 2015)	55.1	65.3	51.1	
Colorization (Zhang et al., 2016a)	61.5	65.6	46.9	35.6
BIGAN (Donahue et al., 2016)	52.3	60.1	46.9	34.9
Jigsaw Puzzles (Noroozi & Favaro, 2016)	-	67.6	53.2	37.6
NAT (Bojanowski & Joulin, 2017)	56.7	65.3	49.4	
Split-Brain (Zhang et al., 2016b)	63.0	67.1	46.7	36.0
ColorProxy (Larsson et al., 2017)		65.9		38.4
Counting (Noroozi et al., 2017)	-	67.7	51.4	36.6
(Ours) RotNet	**70.87**	**72.97**	**54.4**	**39.1**

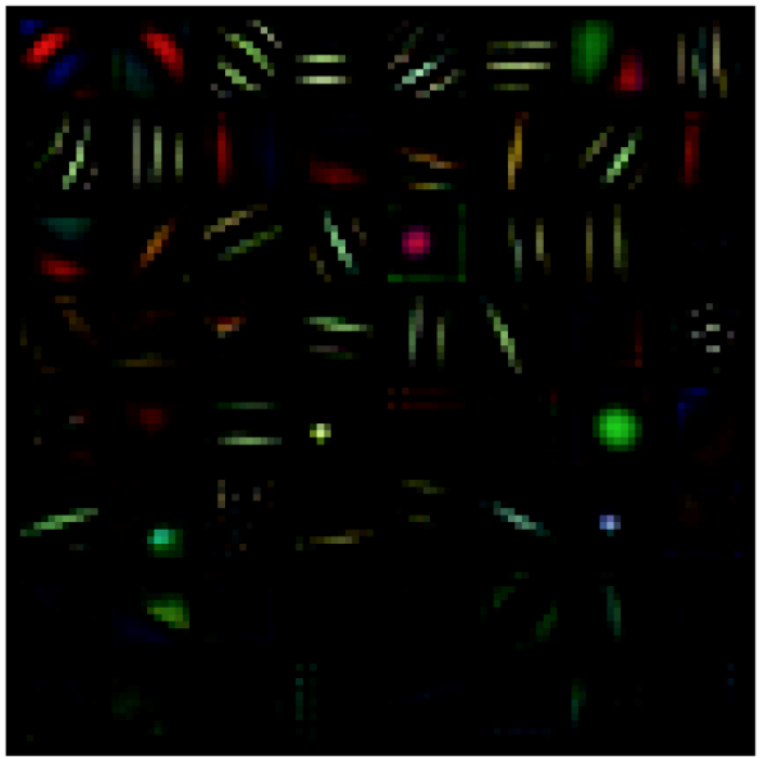
pretrain on annotated images

no pretraining

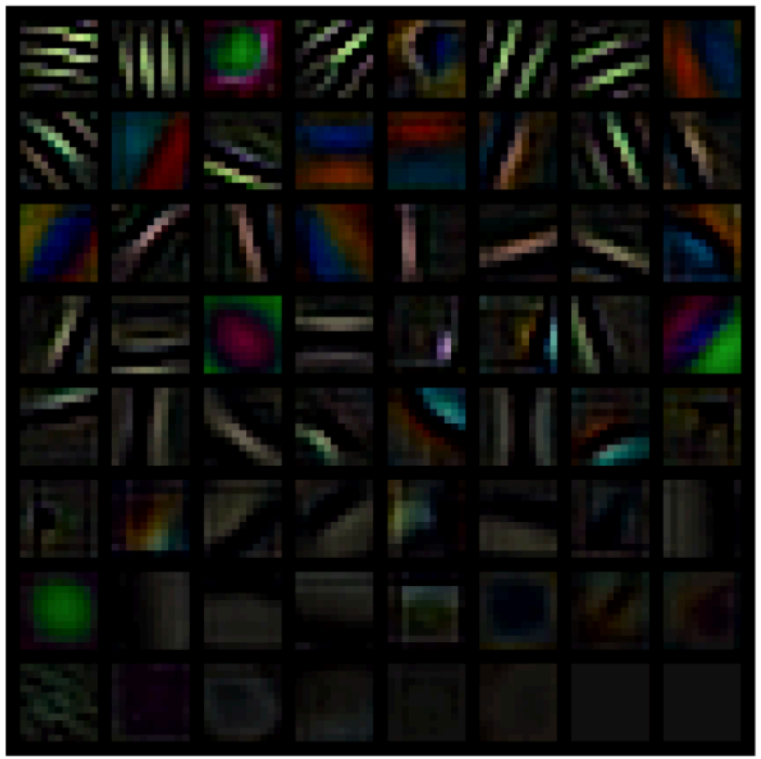
other SSL approaches

SSL with Rotations

What kind of evaluation is this?



(a) **Supervised**

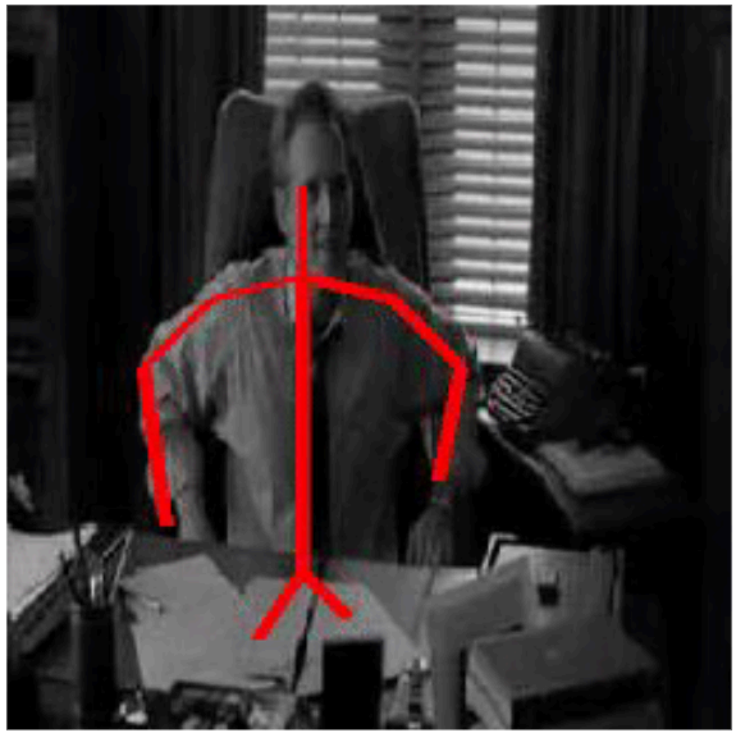


(b) **Self-supervised to recognize rotations**

visualized filter in the first layer of AlexNet

What kind of evaluation is this?

Propagating skeleton keypoints using learned attention.



source: https://research.google/blog/self-supervised-tracking-via-video-colorization/

Invariance vs Covariance

Is it more important that the 2 features of these 2 images can clearly discriminate the rotation, or do we rather care that both are images of a bird?

Invariance vs Covariance

cs.CV] 4 Dec 2019

Self-Supervised Learning of Pretext-Invariant Representations

Ishan Misra Laurens van der Maaten
Facebook AI Research

Abstract

The goal of self-supervised learning from images is to construct image representations that are semantically meaningful via pretext tasks that do not require semantic annotations for a large training set of images. Many pretext tasks lead to representations that are covariant *with image transformations. We argue that, instead, semantic representations ought to be* invariant *under such transformations. Specifically, we develop Pretext-Invariant Representation Learning (PIRL, pronounced as "pearl") that learns invariant representations based on pretext tasks. We use PIRL with a commonly used pretext task that involves solving jigsaw puzzles. We find that PIRL substantially im-*

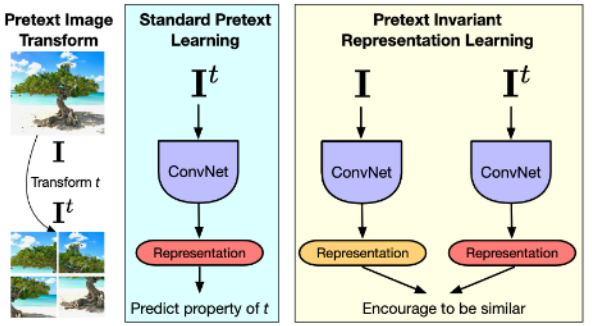
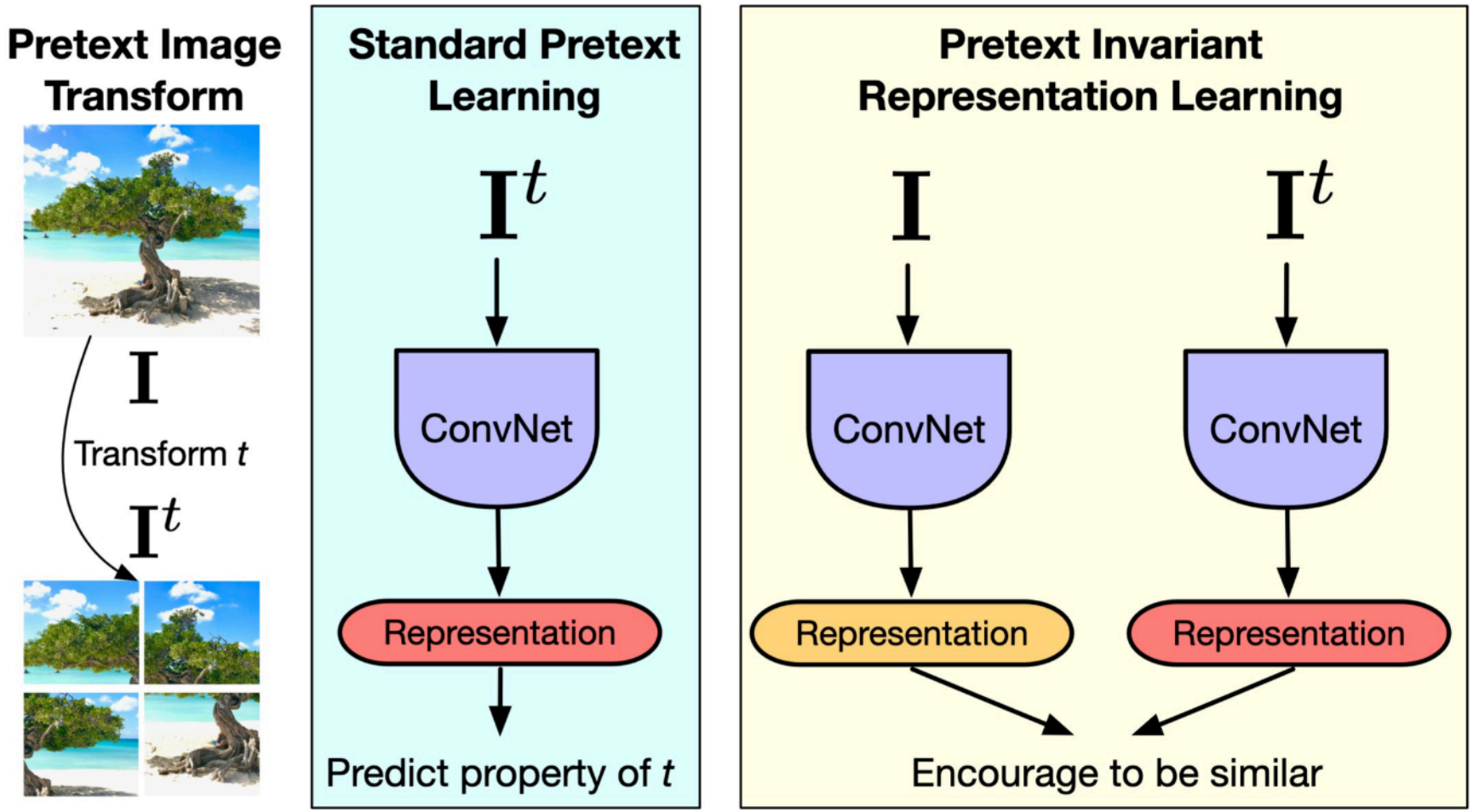


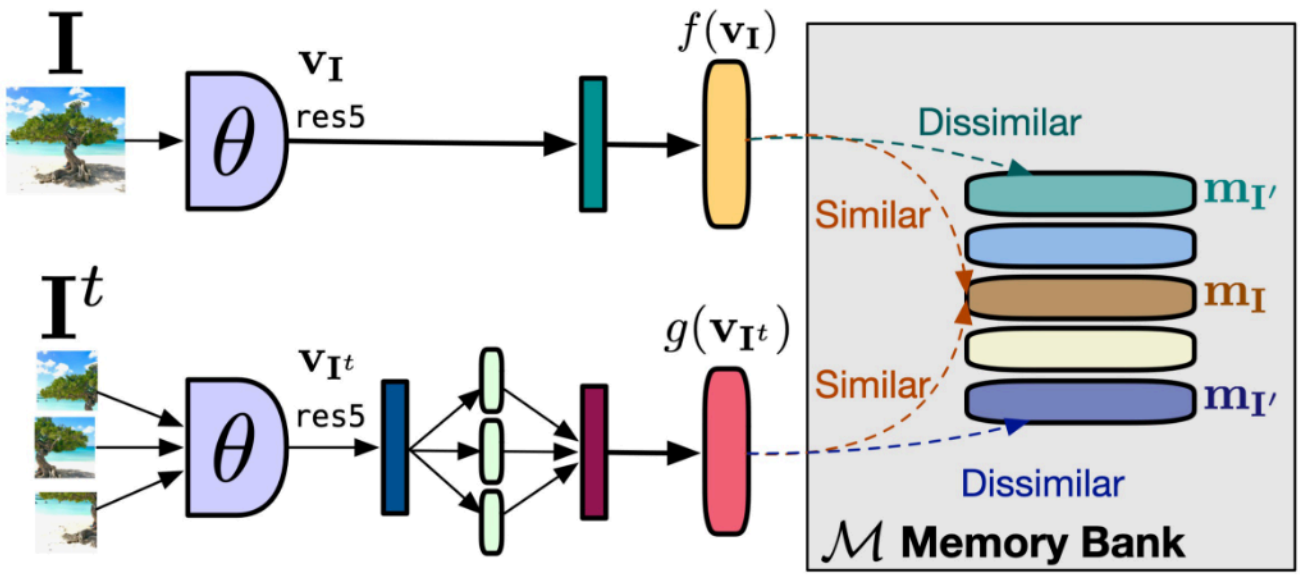
Figure 1: Pretext-Invariant Representation Learning (PIRL). Many pretext tasks for self-supervised learning [18, 46, 76] involve transforming an image **I**, computing a representation of the transformed image, and predicting properties of transformation t from that representation. As a result, the representation must *covary* with the transformation t and may not con-

PIRL: For semantic tasks, we want invariance, not covariance.



How do they align the features?

Encourage similarity with a positive pair and dissimilarity with negative examples.



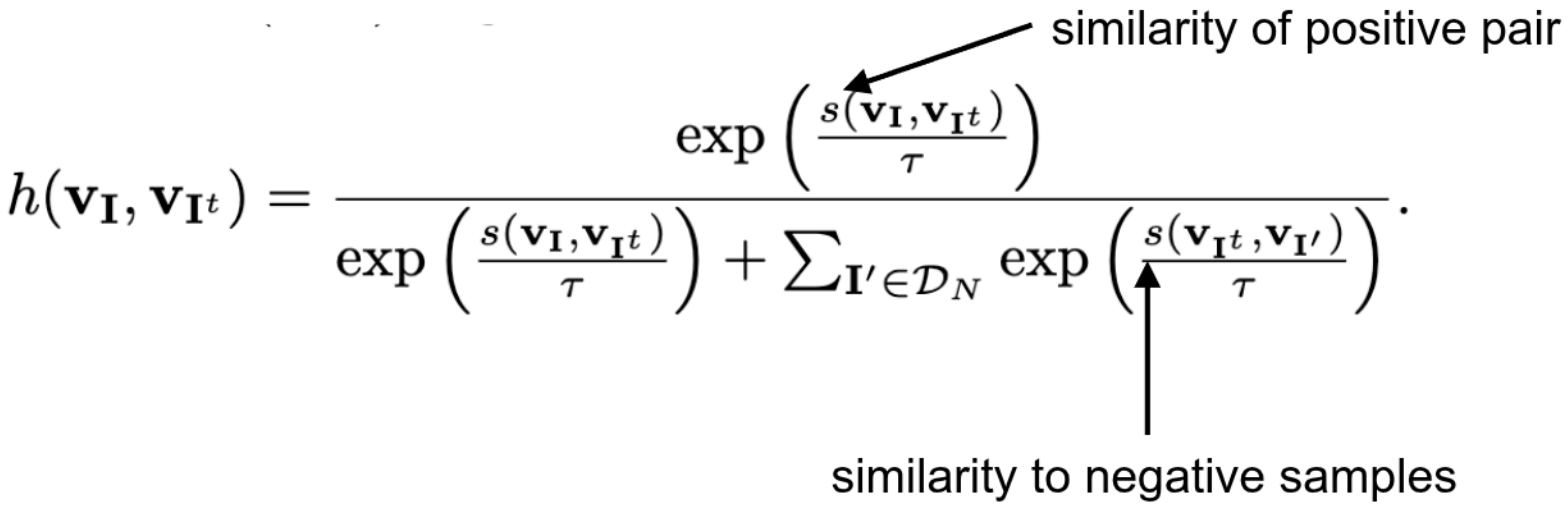
Memory Bank stores features of negative samples, because re-computing would be expensive.

Different Heads to process the jigsaw perturbation.

How do they align the features?

Encourage similarity with a positive pair and dissimilarity with negative examples.

NCE Loss:

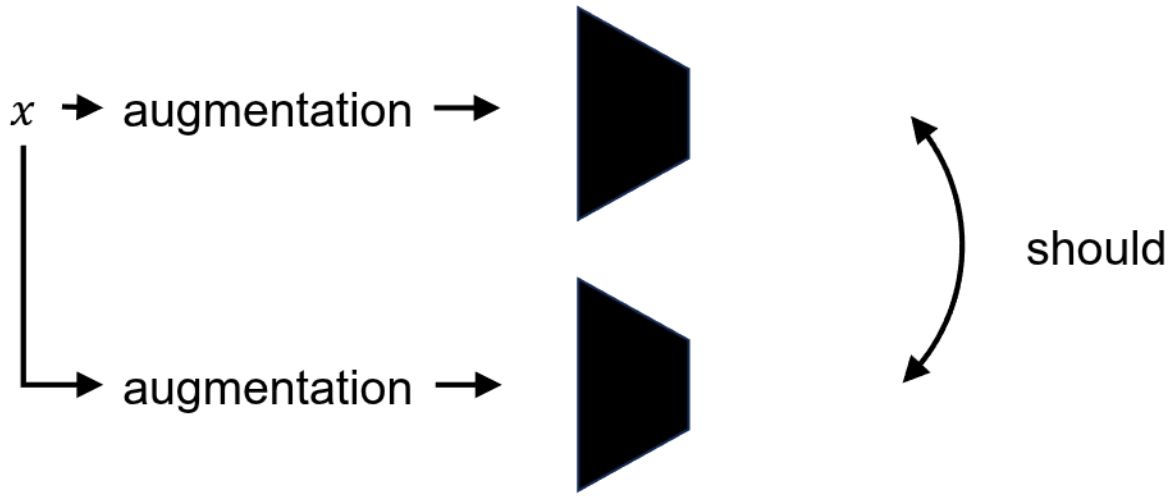


What evaluation method is this?

Method	Parameters	Transfer Dataset			
		ImageNet	VOC07	Places205	iNat.
ResNet-50 using evaluation setup of [19]					
Supervised	25.6M	75.9	87.5	51.5	45.4
Colorization [19]	25.6M	39.6	55.6	37.5	–
Rotation [18]	25.6M	48.9	63.9	41.4	23.0
NPID++ [72]	25.6M	59.0	76.6	46.4	32.4
MoCo [24]	25.6M	60.6	–	–	–
Jigsaw [19]	25.6M	45.7	64.5	41.2	21.3
PIRL (**ours**)	25.6M	**63.6**	**81.1**	**49.8**	**34.1**

Table 2: Image classification with linear models. Image-classification performance on four datasets using the setup of [19]. We train linear classifiers on image representations obtained by self-supervised learners that were pre-trained on ImageNet (without labels). We report the performance for the best-performing layer for each method. We measure mean average precision (mAP) on the VOC07 dataset and top-1 accuracy on all other datasets. Numbers for PIRL, NPID++, Rotation were obtained by us; the other numbers were adopted from their respective papers. Numbers with † were measured using 10-crop evaluation. The best-performing self-supervised learner on each dataset is **boldfaced**.

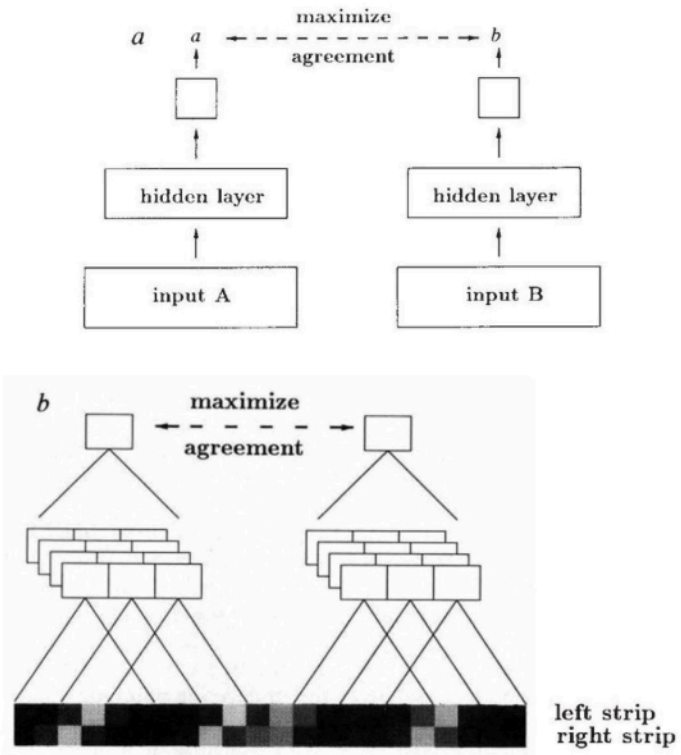
Contrastive SSL with Pretext Augmentations



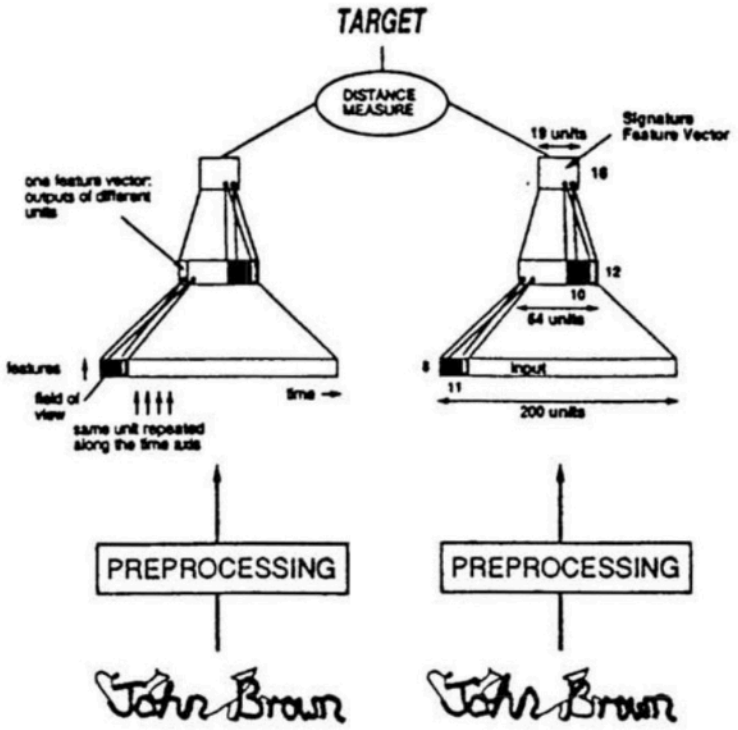
should be the same

This idea goes back to Siamese Networks

Self-organizing neural networks
(Becker & Hinton 1992)

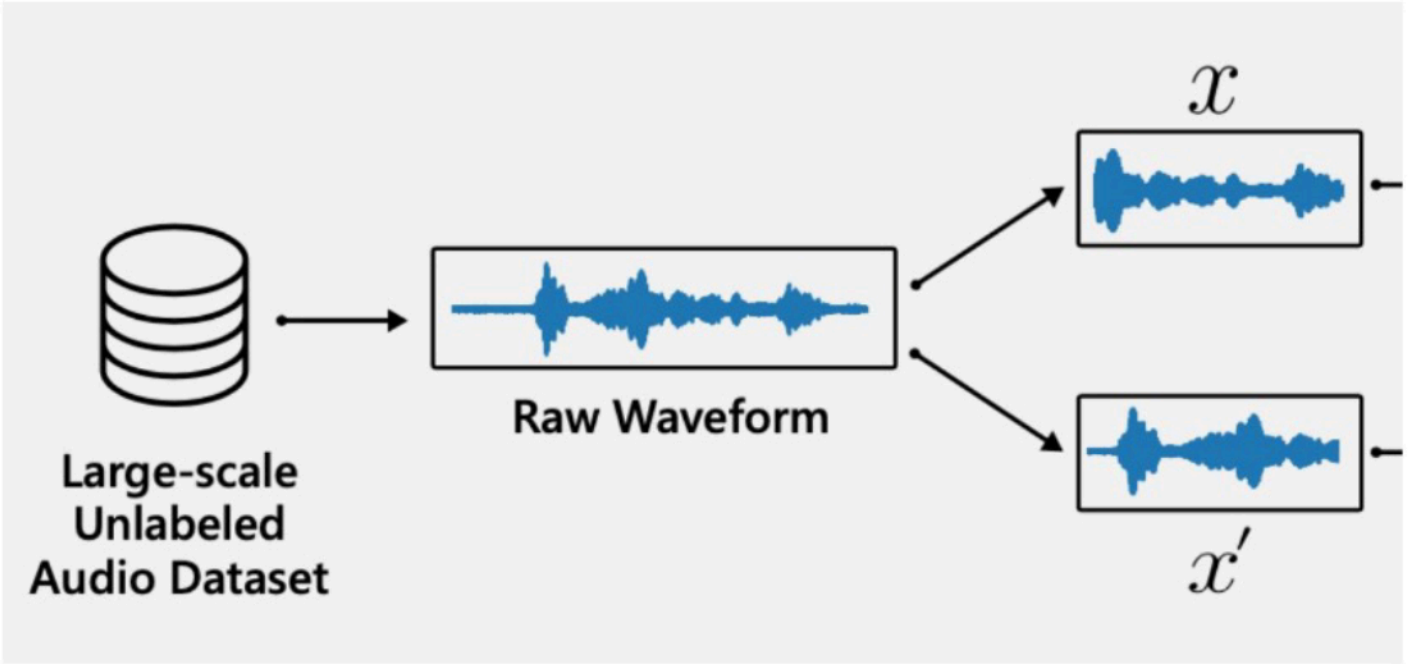


Siamese networks
Bromley et al. (1994)



slide by Weng & Kim, NeurIPS 2021 Tutorial

Audio Example: 2 Clips from same Recording



(Saeed et al. 2021)

How can we train networks to make features similar?

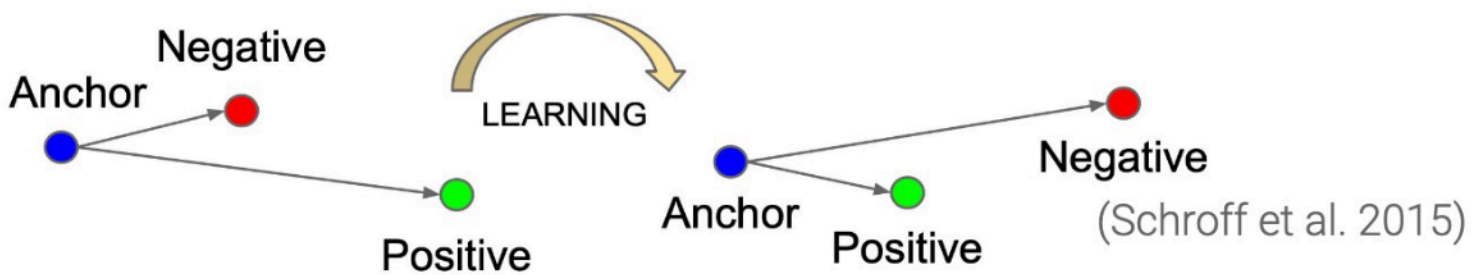
- Triplet Loss
- N-pair Loss
- NCE Loss
- InfoNCE Loss
- Cluster Loss

Triplet Loss

Triplet loss (Schroff et al. 2015): learns to minimize the distance between the anchor **x** and positive **x+** and maximize the distance between the anchor **x** and negative **x-** at the same time.

Given a triplet input $(\mathbf{x}, \mathbf{x}^+, \mathbf{x}^-)$,

$$\mathcal{L}_{\text{triplet}}(\mathbf{x}, \mathbf{x}^+, \mathbf{x}^-) = \sum_{\mathbf{x} \in \mathcal{X}} \max\left(0, \|f(\mathbf{x}) - f(\mathbf{x}^+)\|_2^2 - \|f(\mathbf{x}) - f(\mathbf{x}^-)\|_2^2 + \epsilon\right)$$



(Schroff et al. 2015)

slide by Weng & Kim, NeurIPS 2021 Tutorial

N-Pair Loss

N-pair loss (Sohn 2016) generalizes triplet loss to include comparison with multiple negative samples.

Given one positive and N-1 negative samples, $\{\mathbf{x}, \mathbf{x}^+, \mathbf{x}_1^-, \dots, \mathbf{x}_{N-1}^-\}$

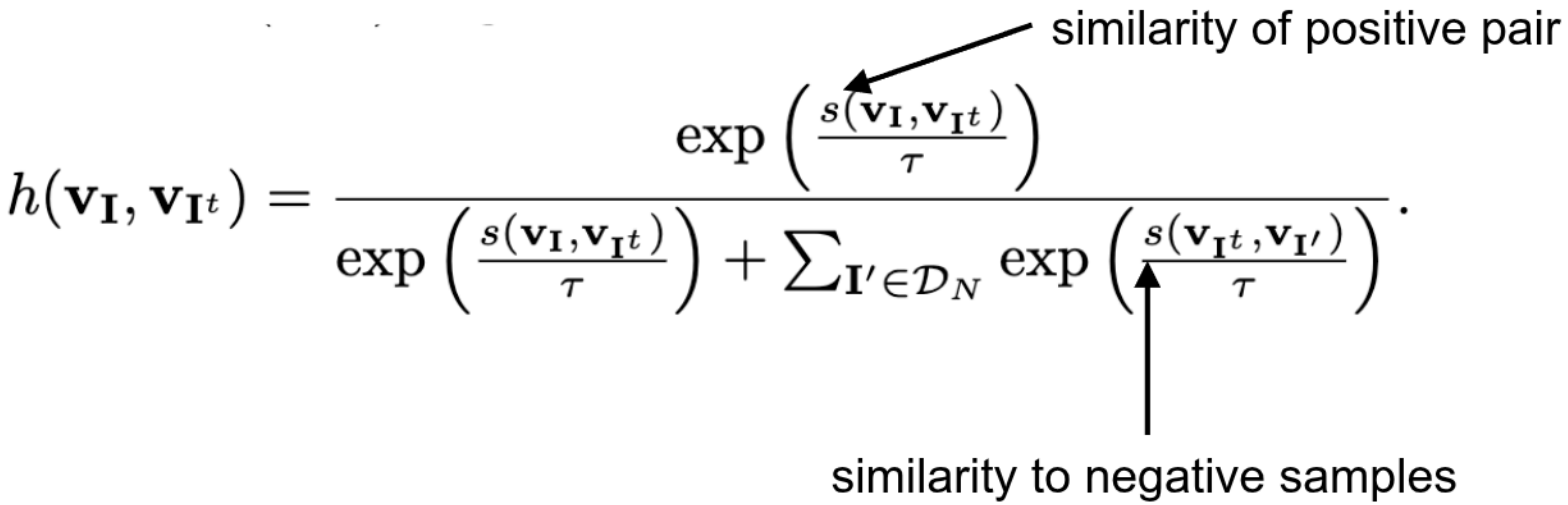
$$\mathcal{L}_{\text{N-pair}}(\mathbf{x}, \mathbf{x}^+, \{\mathbf{x}_i^-\}_{i=1}^{N-1}) = \log\left(1 + \sum_{i=1}^{N-1} \exp(f(\mathbf{x})^\top f(\mathbf{x}_i^-) - f(\mathbf{x})^\top f(\mathbf{x}^+))\right)$$

$$= -\log \frac{\exp(f(\mathbf{x})^\top f(\mathbf{x}^+))}{\exp(f(\mathbf{x})^\top f(\mathbf{x}^+)) + \sum_{i=1}^{N-1} \exp(f(\mathbf{x})^\top f(\mathbf{x}_i^-))}$$

slide by Weng & Kim, NeurIPS 2021 Tutorial

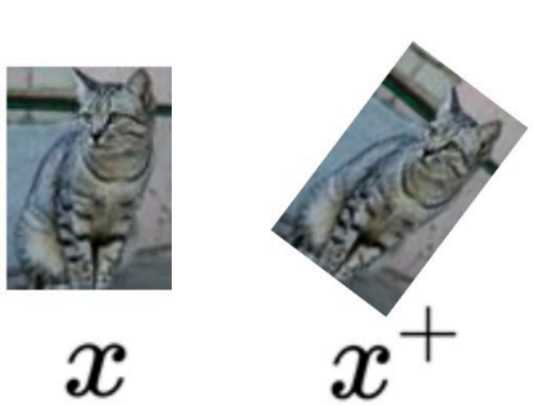
NCE Loss

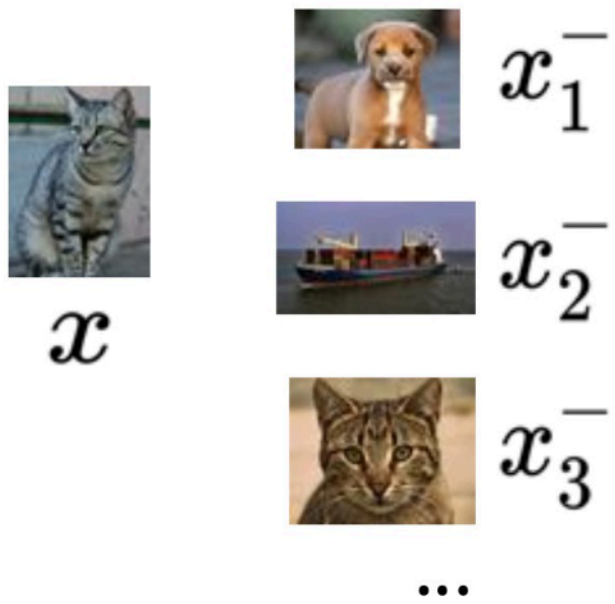
We just saw this in PIRL. It models the probability that the pair [I, aug(I)] appears in the training data X.



InfoNCE Loss

Consider we are given x, and want to classify the augmented x^+ out of many x^-. InfoNCE is the NLL of detecting x^+ correctly with a detector that picks the sample with highest score s.





InfoNCE Loss

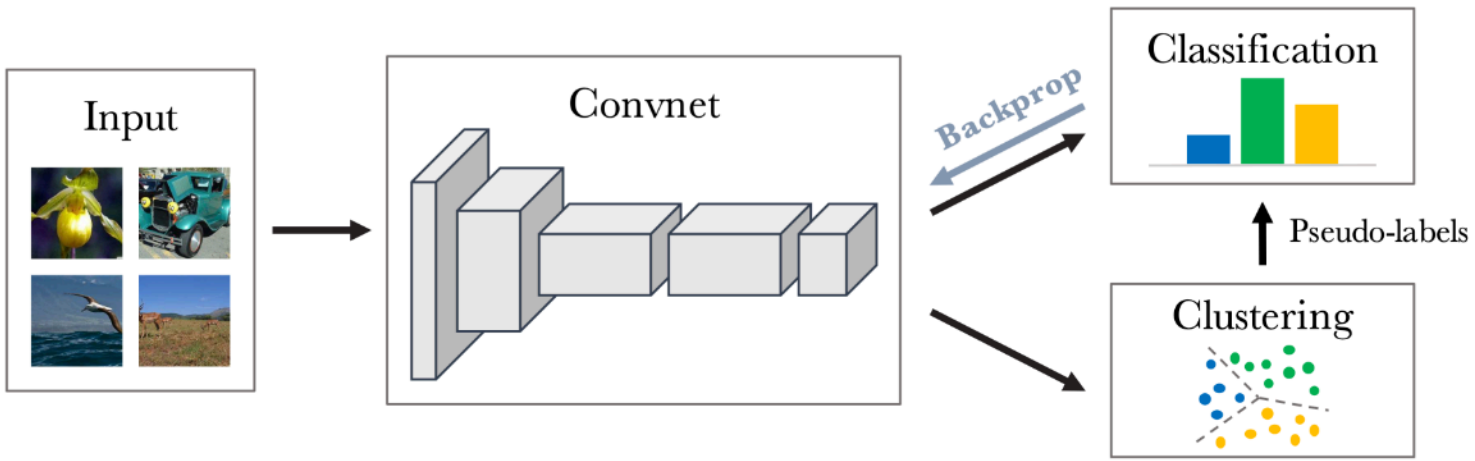
Consider we are given x, and want to classify the augmented x^+ out of many x^-. InfoNCE is the NLL of detecting x^+ correctly with a detector that picks the sample with highest score s.

$$\ell = -\log \frac{\exp(s(x, x^+))}{\exp(s(x, x^+)) + \sum_{k=1}^{K} \exp(s(x, x_k^-))}$$

-> same idea as NCE, but based on cross-entropy

Clustering Loss

Remember the lecture on clustering:



The classification of pseudo-labelled clusters is just another contrastive objective.

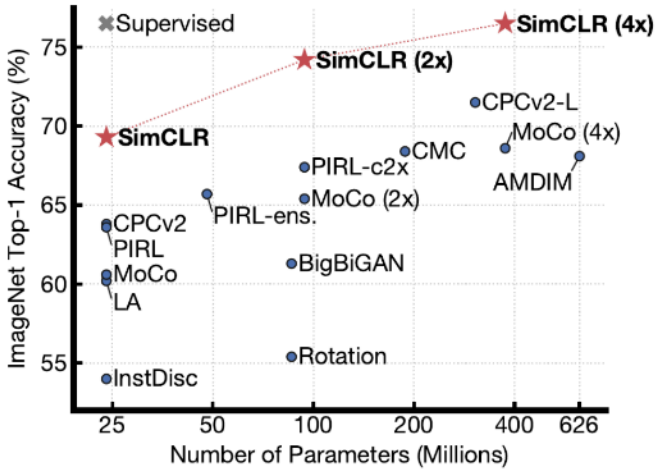
SimCLR

A Simple Framework for Contrastive Learning of Visual Representations

Ting Chen [1] **Simon Kornblith** [1] **Mohammad Norouzi** [1] **Geoffrey Hinton** [1]

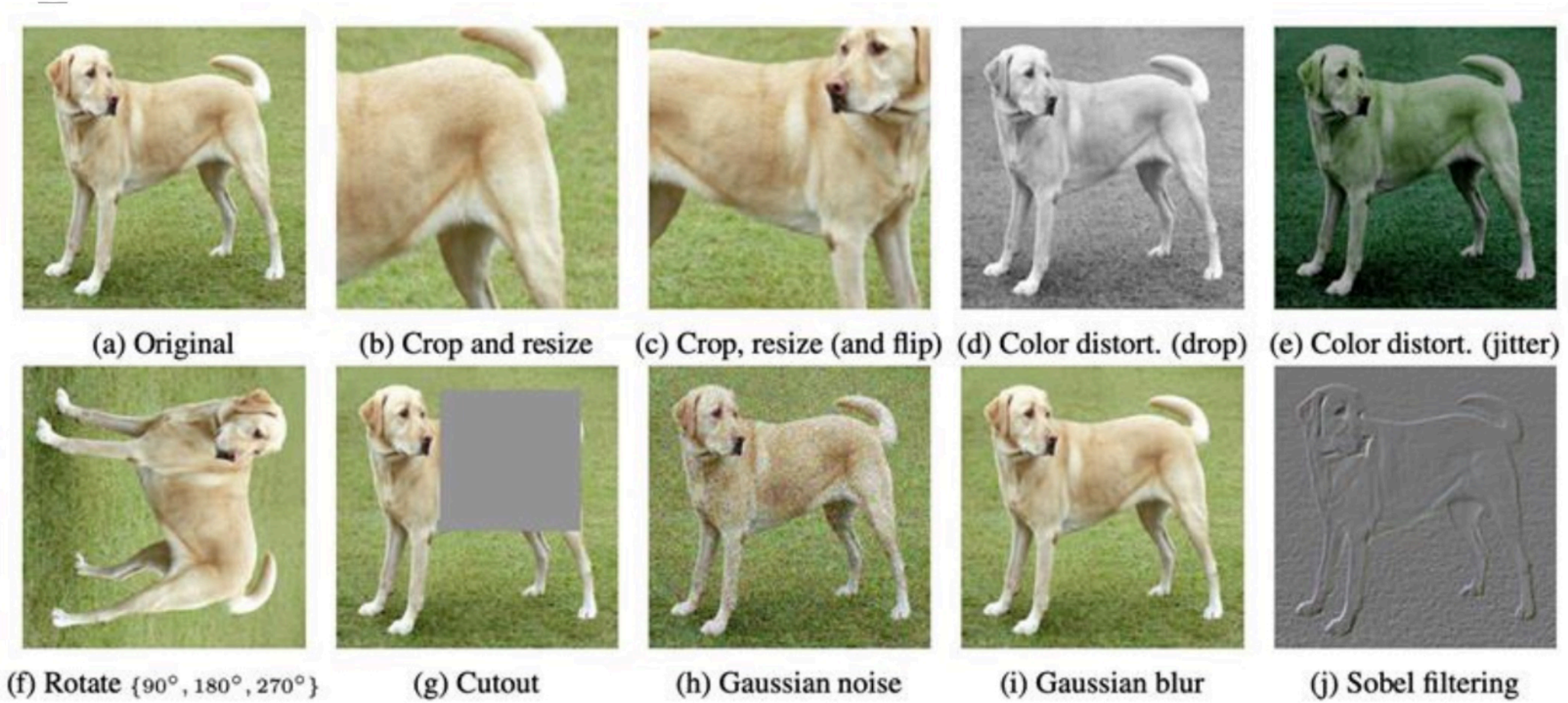
Abstract

This paper presents *SimCLR*: a simple framework for contrastive learning of visual representations. We simplify recently proposed contrastive self-supervised learning algorithms without requiring specialized architectures or a memory bank. In order to understand what enables the contrastive prediction tasks to learn useful representations, we systematically study the major components of our framework. We show that (1) composition of data augmentations plays a critical role in defining effective predictive tasks, (2) introducing a learnable nonlinear transformation between the representation and the contrastive loss substantially improves the quality of the learned representations,



ervised Learning

Mixing many pretext tasks together



Source: Chen et al., 2020

slide by Li et al, Stanford CS231n

Algorithm 1 SimCLR's main learning algorithm.

input: batch size N, constant τ, structure of $f, g, \mathcal{T}$.
for sampled minibatch $\{\boldsymbol{x}_k\}_{k=1}^{N}$ **do**
  for all $k \in \{1, \ldots, N\}$ **do**
    draw two augmentation functions $t \sim \mathcal{T}, t' \sim \mathcal{T}$
    # the first augmentation
    $\tilde{\boldsymbol{x}}_{2k-1} = t(\boldsymbol{x}_k)$
    $\boldsymbol{h}_{2k-1} = f(\tilde{\boldsymbol{x}}_{2k-1})$ # representation
    $\boldsymbol{z}_{2k-1} = g(\boldsymbol{h}_{2k-1})$ # projection
    # the second augmentation
    $\tilde{\boldsymbol{x}}_{2k} = t'(\boldsymbol{x}_k)$
    $\boldsymbol{h}_{2k} = f(\tilde{\boldsymbol{x}}_{2k})$ # representation
    $\boldsymbol{z}_{2k} = g(\boldsymbol{h}_{2k})$ # projection
  end for
  for all $i \in \{1, \ldots, 2N\}$ and $j \in \{1, \ldots, 2N\}$ **do**
    $s_{i,j} = \boldsymbol{z}_i^\top \boldsymbol{z}_j / (\|\boldsymbol{z}_i\| \|\boldsymbol{z}_j\|)$ # pairwise similarity
  end for
  define $\ell(i, j)$ **as** $\ell(i, j) = -\log \frac{\exp(s_{i,j}/\tau)}{\sum_{k=1}^{2N} \mathbb{1}_{[k \neq i]} \exp(s_{i,k}/\tau)}$
  $\mathcal{L} = \frac{1}{2N} \sum_{k=1}^{N} [\ell(2k-1, 2k) + \ell(2k, 2k-1)]$
  update networks f and g to minimize $\mathcal{L}$
end for
return encoder network $f(\cdot)$, and throw away $g(\cdot)$

generate a positive pair from 2 augmentations

Chen et al, 2020

Algorithm 1 SimCLR's main learning algorithm.

input: batch size N, constant τ, structure of $f, g, \mathcal{T}$.
for sampled minibatch $\{\boldsymbol{x}_k\}_{k=1}^{N}$ **do**
  for all $k \in \{1, \ldots, N\}$ **do**
    draw two augmentation functions $t \sim \mathcal{T}, t' \sim \mathcal{T}$
    # the first augmentation
    $\tilde{\boldsymbol{x}}_{2k-1} = t(\boldsymbol{x}_k)$
    $\boldsymbol{h}_{2k-1} = f(\tilde{\boldsymbol{x}}_{2k-1})$ # representation
    $\boldsymbol{z}_{2k-1} = g(\boldsymbol{h}_{2k-1})$ # projection
    # the second augmentation
    $\tilde{\boldsymbol{x}}_{2k} = t'(\boldsymbol{x}_k)$
    $\boldsymbol{h}_{2k} = f(\tilde{\boldsymbol{x}}_{2k})$ # representation
    $\boldsymbol{z}_{2k} = g(\boldsymbol{h}_{2k})$ # projection
  end for
  for all $i \in \{1, \ldots, 2N\}$ and $j \in \{1, \ldots, 2N\}$ **do**
    $s_{i,j} = \boldsymbol{z}_i^\top \boldsymbol{z}_j / (\|\boldsymbol{z}_i\| \|\boldsymbol{z}_j\|)$ # pairwise similarity
  end for
  define $\ell(i, j)$ **as** $\ell(i, j) = -\log \frac{\exp(s_{i,j}/\tau)}{\sum_{k=1}^{2N} \mathbb{1}_{[k \neq i]} \exp(s_{i,k}/\tau)}$
  $\mathcal{L} = \frac{1}{2N} \sum_{k=1}^{N} [\ell(2k-1, 2k) + \ell(2k, 2k-1)]$
  update networks f and g to minimize $\mathcal{L}$
end for
return encoder network $f(\cdot)$, and throw away $g(\cdot)$

Chen et al, 2020

generate a positive pair from 2 augmentations

project the feature into a contrastive space

Algorithm 1 SimCLR's main learning algorithm.

input: batch size N, constant τ, structure of $f, g, \mathcal{T}$.
for sampled minibatch $\{\boldsymbol{x}_k\}_{k=1}^{N}$ **do**
  for all $k \in \{1, \ldots, N\}$ **do**
    draw two augmentation functions $t \sim \mathcal{T}, t' \sim \mathcal{T}$
    # the first augmentation
    $\tilde{\boldsymbol{x}}_{2k-1} = t(\boldsymbol{x}_k)$
    $\boldsymbol{h}_{2k-1} = f(\tilde{\boldsymbol{x}}_{2k-1})$ # representation
    $\boldsymbol{z}_{2k-1} = g(\boldsymbol{h}_{2k-1})$ # projection
    # the second augmentation
    $\tilde{\boldsymbol{x}}_{2k} = t'(\boldsymbol{x}_k)$
    $\boldsymbol{h}_{2k} = f(\tilde{\boldsymbol{x}}_{2k})$ # representation
    $\boldsymbol{z}_{2k} = g(\boldsymbol{h}_{2k})$ # projection
  end for
  for all $i \in \{1, \ldots, 2N\}$ and $j \in \{1, \ldots, 2N\}$ **do**
    $s_{i,j} = \boldsymbol{z}_i^\top \boldsymbol{z}_j / (\|\boldsymbol{z}_i\| \|\boldsymbol{z}_j\|)$ # pairwise similarity
  end for
  define $\ell(i, j)$ **as** $\ell(i,j) = -\log \frac{\exp(s_{i,j}/\tau)}{\sum_{k=1}^{2N} \mathbb{1}_{[k \neq i]} \exp(s_{i,k}/\tau)}$
  $\mathcal{L} = \frac{1}{2N} \sum_{k=1}^{N} [\ell(2k-1, 2k) + \ell(2k, 2k-1)]$
  update networks f and g to minimize $\mathcal{L}$
end for
return encoder network $f(\cdot)$, and throw away $g(\cdot)$

generate a positive pair from 2 augmentations

project the feature into a contrastive space

apply InfoNCE loss

Chen et al, 2020

Algorithm 1 SimCLR's main learning algorithm.

input: batch size N, constant τ, structure of $f, g, \mathcal{T}$.
for sampled minibatch $\{\boldsymbol{x}_k\}_{k=1}^{N}$ **do**
  for all $k \in \{1, \ldots, N\}$ **do**
    draw two augmentation functions $t \sim \mathcal{T}, t' \sim \mathcal{T}$
    # the first augmentation
    $\tilde{\boldsymbol{x}}_{2k-1} = t(\boldsymbol{x}_k)$
    $\boldsymbol{h}_{2k-1} = f(\tilde{\boldsymbol{x}}_{2k-1})$ # representation
    $\boldsymbol{z}_{2k-1} = g(\boldsymbol{h}_{2k-1})$ # projection
    # the second augmentation
    $\tilde{\boldsymbol{x}}_{2k} = t'(\boldsymbol{x}_k)$
    $\boldsymbol{h}_{2k} = f(\tilde{\boldsymbol{x}}_{2k})$ # representation
    $\boldsymbol{z}_{2k} = g(\boldsymbol{h}_{2k})$ # projection
  end for
  for all $i \in \{1, \ldots, 2N\}$ and $j \in \{1, \ldots, 2N\}$ **do**
    $s_{i,j} = \boldsymbol{z}_i^\top \boldsymbol{z}_j / (\|\boldsymbol{z}_i\| \|\boldsymbol{z}_j\|)$ # pairwise similarity
  end for
  define $\ell(i, j)$ **as** $\ell(i, j) = -\log \frac{\exp(s_{i,j}/\tau)}{\sum_{k=1}^{2N} \mathbb{1}_{[k \neq i]} \exp(s_{i,k}/\tau)}$
  $\mathcal{L} = \frac{1}{2N} \sum_{k=1}^{N} [\ell(2k-1, 2k) + \ell(2k, 2k-1)]$
  update networks f and g to minimize $\mathcal{L}$
end for
return encoder network $f(\cdot)$, and throw away $g(\cdot)$

Chen et al, 2020

generate a positive pair from 2 augmentations

project the feature into a contrastive space

apply InfoNCE loss

iterate through entire batch, always using all other samples in the batch as negatives

SimCLR Results

Method	Architecture	Label fraction 1%	Label fraction 10%
		Top 5	Top 5
Supervised baseline	ResNet-50	48.4	80.4
Methods using other label-propagation:			
Pseudo-label	ResNet-50	51.6	82.4
VAT+Entropy Min.	ResNet-50	47.0	83.4
UDA (w. RandAug)	ResNet-50	-	88.5
FixMatch (w. RandAug)	ResNet-50	-	89.1
S4L (Rot+VAT+En. M.)	ResNet-50 (4×)	-	91.2
Methods using representation learning only:			
InstDisc	ResNet-50	39.2	77.4
BigBiGAN	RevNet-50 (4×)	55.2	78.8
PIRL	ResNet-50	57.2	83.8
CPC v2	ResNet-161(*)	77.9	91.2
SimCLR (ours)	ResNet-50	75.5	87.8
SimCLR (ours)	ResNet-50 (2×)	83.0	91.2
SimCLR (ours)	ResNet-50 (4×)	**85.8**	**92.6**

Table 7. ImageNet accuracy of models trained with few labels.

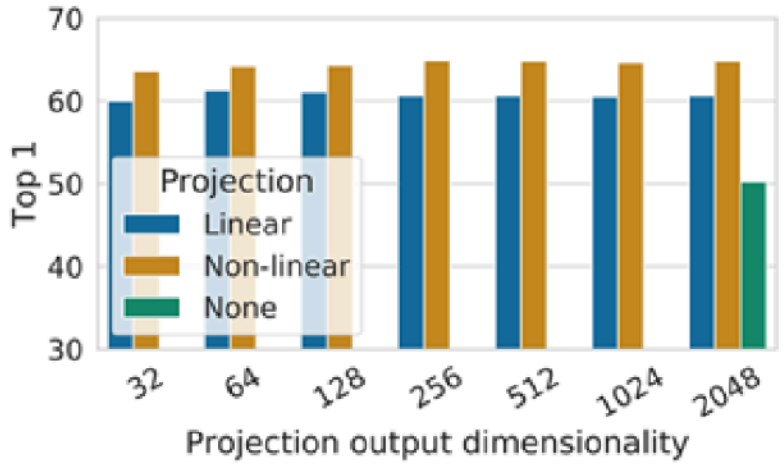
Train feature encoder on ImageNet (entire training set) using SimCLR.

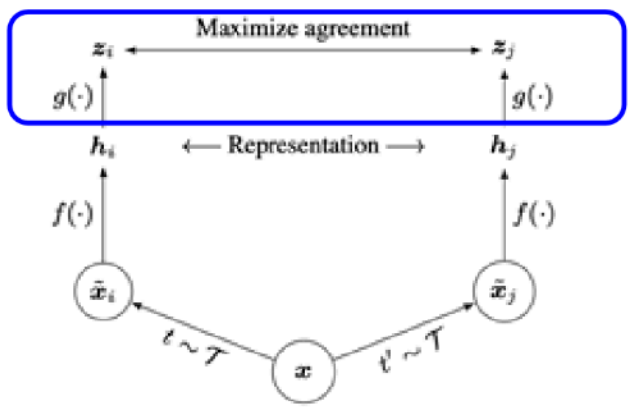
Finetune the encoder with 1% / 10% of labeled data on ImageNet.

Source: Chen et al., 2020

slide by Li et al, Stanford CS231n

Why a projection head?





Linear / non-linear projection heads improve representation learning.

A possible explanation:

- contrastive learning objective may discard useful information for downstream tasks
- representation space z is trained to be invariant to data transformation.
- by leveraging the projection head g(·), more information can be preserved in the h representation space

Source: Chen et al., 2020

slide by Li et al, Stanford CS231n

SimCLR needs a large batch size

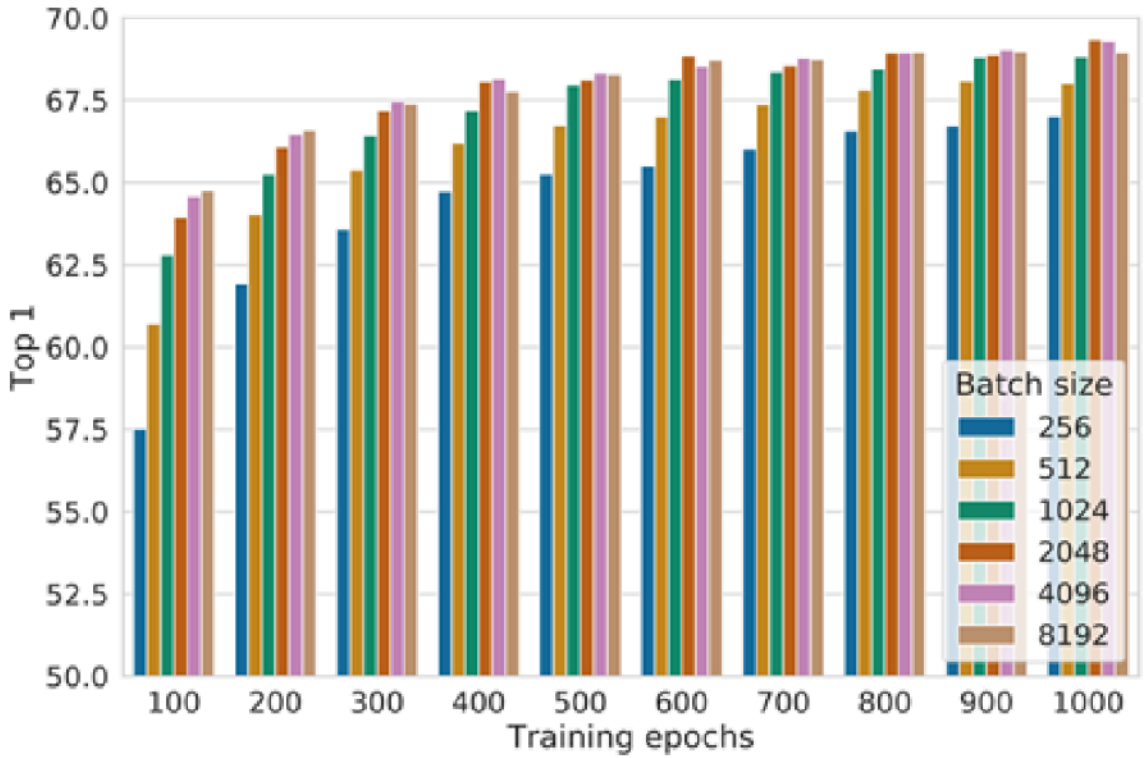


Figure 9. Linear evaluation models (ResNet-50) trained with different batch size and epochs. Each bar is a single run from scratch.[10]

Large training batch size is crucial for SimCLR!

Large batch size causes large memory footprint during backpropagation: requires distributed training on TPUs (ImageNet experiments)

$$MI[f(x), f(x^+)] - \log(N) \geq -L$$

Source: Chen et al., 2020

slide by Li et al, Stanford CS231n

MoCo

This CVPR 2020 paper is the Open Access version, provided by the Computer Vision Foundation.
Except for this watermark, it is identical to the accepted version;
the final published version of the proceedings is available on IEEE Xplore.

Momentum Contrast for Unsupervised Visual Representation Learning

Kaiming He Haoqi Fan Yuxin Wu Saining Xie Ross Girshick

Facebook AI Research (FAIR)

Code: https://github.com/facebookresearch/moco

Abstract

We present Momentum Contrast (MoCo) for unsupervised visual representation learning. From a perspective on contrastive learning [29] as dictionary look-up, we build a dynamic dictionary with a queue and a moving-averaged encoder. This enables building a large and consistent dictionary on-the-fly that facilitates contrastive unsupervised learning. MoCo provides competitive results under the common linear protocol on ImageNet classification. More importantly, the representations learned by MoCo transfer well to downstream tasks. MoCo can ***outperform*** *its supervised pre-training counterpart in 7 detection/segmentation tasks on PASCAL VOC, COCO, and other datasets, sometimes surpassing it by large margins. This suggests that the gap between unsupervised and supervised representation learning has been largely closed in many vision tasks.*

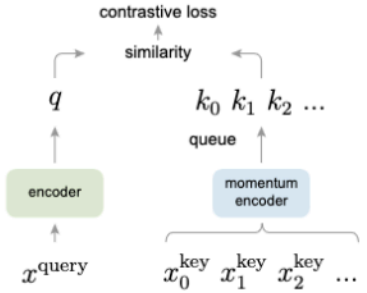
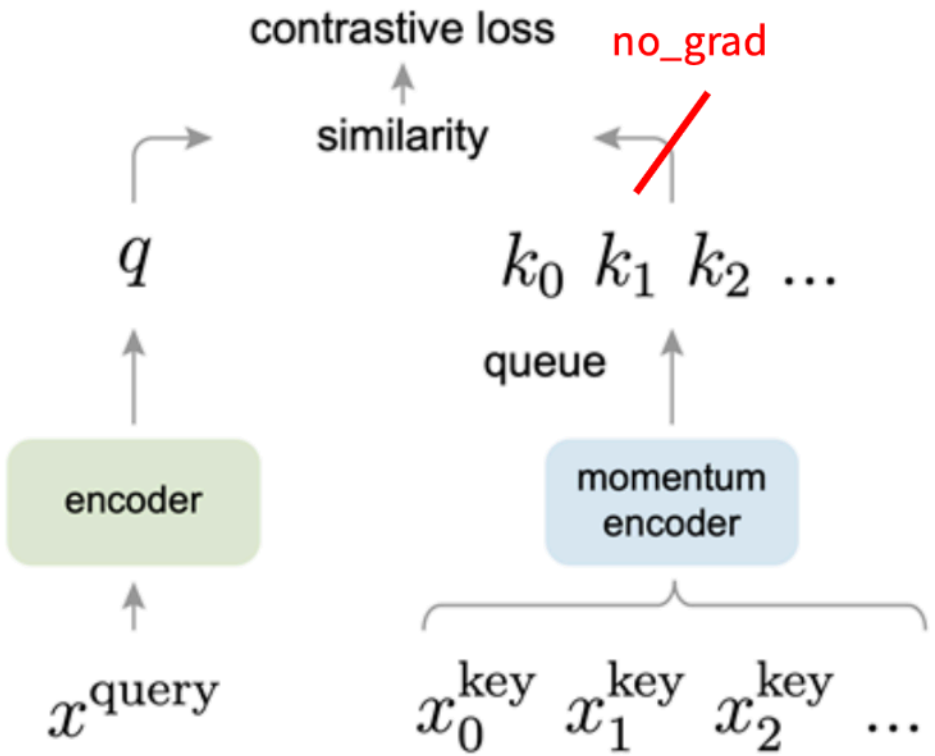


Figure 1. Momentum Contrast (MoCo) trains a visual representation encoder by matching an encoded query q to a dictionary of encoded keys using a contrastive loss. The dictionary keys $\{k_0, k_1, k_2, ...\}$ are defined on-the-fly by a set of data samples. The dictionary is built as a queue, with the current mini-batch enqueued and the oldest mini-batch dequeued, decoupling it from the mini-batch size. The keys are encoded by a slowly progressing

Momentum Contrast (MoCo)



Key differences to SimCLR:

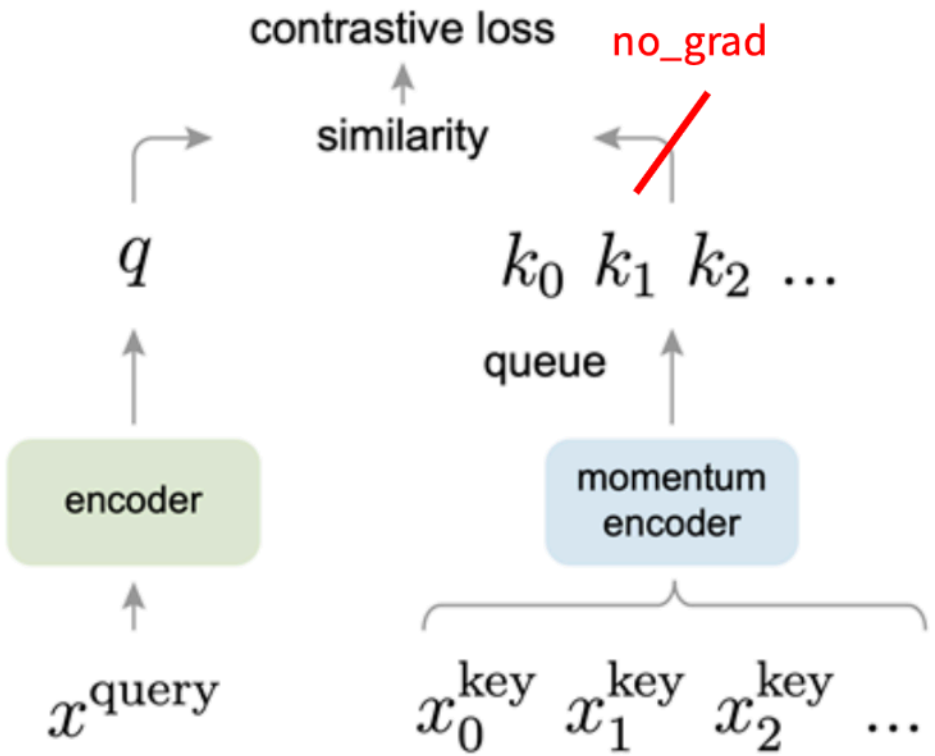
- Keep a running queue of keys (negative samples).
- Compute gradients and update the encoder only through the queries.
- Decouple min-batch size with the number of keys: can support a large number of negative samples.
- The key encoder is slowly progressing through the momentum update rules:

$$\theta_{\text{k}} \leftarrow m\theta_{\text{k}} + (1-m)\theta_{\text{q}}$$

Source: He et al., 2020

slide by Li et al, Stanford CS231n

Momentum Contrast (MoCo)



this will proof very useful!
usually now called exponential moving average

- The key encoder is slowly progressing through the momentum update rules:

$$\theta_{\text{k}} \leftarrow m\theta_{\text{k}} + (1-m)\theta_{\text{q}}$$

Source: He et al., 2020

slide by Li et al, Stanford CS231n

Algorithm 1 Pseudocode of MoCo in a PyTorch-like style.

```
# f_q, f_k: encoder networks for query and key
# queue: dictionary as a queue of K keys (CxK)
# m: momentum
# t: temperature

f_k.params = f_q.params # initialize
for x in loader: # load a minibatch x with N samples
   x_q = aug(x) # a randomly augmented version
   x_k = aug(x) # another randomly augmented version

   q = f_q.forward(x_q) # queries: NxC
   k = f_k.forward(x_k) # keys: NxC
   k = k.detach() # no gradient to keys

   # positive logits: Nx1
   l_pos = bmm(q.view(N,1,C), k.view(N,C,1))

   # negative logits: NxK
   l_neg = mm(q.view(N,C), queue.view(C,K))

   # logits: Nx(1+K)
   logits = cat([l_pos, l_neg], dim=1)

   # contrastive loss, Eqn.(1)
   labels = zeros(N) # positives are the 0-th
   loss = CrossEntropyLoss(logits/t, labels)

   # SGD update: query network
   loss.backward()
   update(f_q.params)

   # momentum update: key network
   f_k.params = m*f_k.params+(1-m)*f_q.params

   # update dictionary
   enqueue(queue, k) # enqueue the current minibatch
   dequeue(queue) # dequeue the earliest minibatch
```

bmm: batch matrix multiplication; mm: matrix multiplication; cat: concatenation.

generate a positive pair from 2 augmentations

Algorithm 1 Pseudocode of MoCo in a PyTorch-like style.

```
# f_q, f_k: encoder networks for query and key
# queue: dictionary as a queue of K keys (CxK)
# m: momentum
# t: temperature

f_k.params = f_q.params # initialize
for x in loader: # load a minibatch x with N samples
   x_q = aug(x) # a randomly augmented version
   x_k = aug(x) # another randomly augmented version

   q = f_q.forward(x_q) # queries: NxC
   k = f_k.forward(x_k) # keys: NxC
   k = k.detach() # no gradient to keys

   # positive logits: Nx1
   l_pos = bmm(q.view(N,1,C), k.view(N,C,1))

   # negative logits: NxK
   l_neg = mm(q.view(N,C), queue.view(C,K))

   # logits: Nx(1+K)
   logits = cat([l_pos, l_neg], dim=1)

   # contrastive loss, Eqn.(1)
   labels = zeros(N) # positives are the 0-th
   loss = CrossEntropyLoss(logits/t, labels)

   # SGD update: query network
   loss.backward()
   update(f_q.params)

   # momentum update: key network
   f_k.params = m*f_k.params+(1-m)*f_q.params

   # update dictionary
   enqueue(queue, k) # enqueue the current minibatch
   dequeue(queue) # dequeue the earliest minibatch
```

bmm: batch matrix multiplication; mm: matrix multiplication; cat: concatenation.

generate a positive pair from 2 augmentations

no gradient through the key

Algorithm 1 Pseudocode of MoCo in a PyTorch-like style.

```
# f_q, f_k: encoder networks for query and key
# queue: dictionary as a queue of K keys (CxK)
# m: momentum
# t: temperature

f_k.params = f_q.params # initialize
for x in loader: # load a minibatch x with N samples
   x_q = aug(x) # a randomly augmented version
   x_k = aug(x) # another randomly augmented version

   q = f_q.forward(x_q) # queries: NxC
   k = f_k.forward(x_k) # keys: NxC
   k = k.detach() # no gradient to keys

   # positive logits: Nx1
   l_pos = bmm(q.view(N,1,C), k.view(N,C,1))

   # negative logits: NxK
   l_neg = mm(q.view(N,C), queue.view(C,K))

   # logits: Nx(1+K)
   logits = cat([l_pos, l_neg], dim=1)

   # contrastive loss, Eqn.(1)
   labels = zeros(N) # positives are the 0-th
   loss = CrossEntropyLoss(logits/t, labels)

   # SGD update: query network
   loss.backward()
   update(f_q.params)

   # momentum update: key network
   f_k.params = m*f_k.params+(1-m)*f_q.params

   # update dictionary
   enqueue(queue, k) # enqueue the current minibatch
   dequeue(queue) # dequeue the earliest minibatch
```

bmm: batch matrix multiplication; mm: matrix multiplication; cat: concatenation.

generate a positive pair from 2 augmentations

no gradient through the key

embeddings from whole queue serve as negatives

Algorithm 1 Pseudocode of MoCo in a PyTorch-like style.

```
# f_q, f_k: encoder networks for query and key
# queue: dictionary as a queue of K keys (CxK)
# m: momentum
# t: temperature

f_k.params = f_q.params # initialize
for x in loader: # load a minibatch x with N samples
   x_q = aug(x) # a randomly augmented version
   x_k = aug(x) # another randomly augmented version

   q = f_q.forward(x_q) # queries: NxC
   k = f_k.forward(x_k) # keys: NxC
   k = k.detach() # no gradient to keys

   # positive logits: Nx1
   l_pos = bmm(q.view(N,1,C), k.view(N,C,1))

   # negative logits: NxK
   l_neg = mm(q.view(N,C), queue.view(C,K))

   # logits: Nx(1+K)
   logits = cat([l_pos, l_neg], dim=1)

   # contrastive loss, Eqn.(1)
   labels = zeros(N) # positives are the 0-th
   loss = CrossEntropyLoss(logits/t, labels)

   # SGD update: query network
   loss.backward()
   update(f_q.params)

   # momentum update: key network
   f_k.params = m*f_k.params+(1-m)*f_q.params

   # update dictionary
   enqueue(queue, k) # enqueue the current minibatch
   dequeue(queue) # dequeue the earliest minibatch
```

bmm: batch matrix multiplication; mm: matrix multiplication; cat: concatenation.

generate a positive pair from 2 augmentations

no gradient through the key

embeddings from whole queue serve as negatives

InfoNCE loss

Algorithm 1 Pseudocode of MoCo in a PyTorch-like style.

```
# f_q, f_k: encoder networks for query and key
# queue: dictionary as a queue of K keys (CxK)
# m: momentum
# t: temperature

f_k.params = f_q.params # initialize
for x in loader: # load a minibatch x with N samples
   x_q = aug(x) # a randomly augmented version
   x_k = aug(x) # another randomly augmented version

   q = f_q.forward(x_q) # queries: NxC
   k = f_k.forward(x_k) # keys: NxC
   k = k.detach() # no gradient to keys

   # positive logits: Nx1
   l_pos = bmm(q.view(N,1,C), k.view(N,C,1))

   # negative logits: NxK
   l_neg = mm(q.view(N,C), queue.view(C,K))

   # logits: Nx(1+K)
   logits = cat([l_pos, l_neg], dim=1)

   # contrastive loss, Eqn.(1)
   labels = zeros(N) # positives are the 0-th
   loss = CrossEntropyLoss(logits/t, labels)

   # SGD update: query network
   loss.backward()
   update(f_q.params)

   # momentum update: key network
   f_k.params = m*f_k.params+(1-m)*f_q.params

   # update dictionary
   enqueue(queue, k) # enqueue the current minibatch
   dequeue(queue) # dequeue the earliest minibatch
```

bmm: batch matrix multiplication; mm: matrix multiplication; cat: concatenation.

generate a positive pair from 2 augmentations

no gradient through the key

embeddings from whole queue serve as negatives

InfoNCE loss

momentum update

Algorithm 1 Pseudocode of MoCo in a PyTorch-like style.

```
# f_q, f_k: encoder networks for query and key
# queue: dictionary as a queue of K keys (CxK)
# m: momentum
# t: temperature

f_k.params = f_q.params # initialize
for x in loader: # load a minibatch x with N samples
   x_q = aug(x) # a randomly augmented version
   x_k = aug(x) # another randomly augmented version

   q = f_q.forward(x_q) # queries: NxC
   k = f_k.forward(x_k) # keys: NxC
   k = k.detach() # no gradient to keys

   # positive logits: Nx1
   l_pos = bmm(q.view(N,1,C), k.view(N,C,1))

   # negative logits: NxK
   l_neg = mm(q.view(N,C), queue.view(C,K))

   # logits: Nx(1+K)
   logits = cat([l_pos, l_neg], dim=1)

   # contrastive loss, Eqn.(1)
   labels = zeros(N) # positives are the 0-th
   loss = CrossEntropyLoss(logits/t, labels)

   # SGD update: query network
   loss.backward()
   update(f_q.params)

   # momentum update: key network
   f_k.params = m*f_k.params+(1-m)*f_q.params

   # update dictionary
   enqueue(queue, k) # enqueue the current minibatch
   dequeue(queue) # dequeue the earliest minibatch
```

bmm: batch matrix multiplication; mm: matrix multiplication; cat: concatenation.

generate a positive pair from 2 augmentations

no gradient through the key

embeddings from whole queue serve as negatives

InfoNCE loss

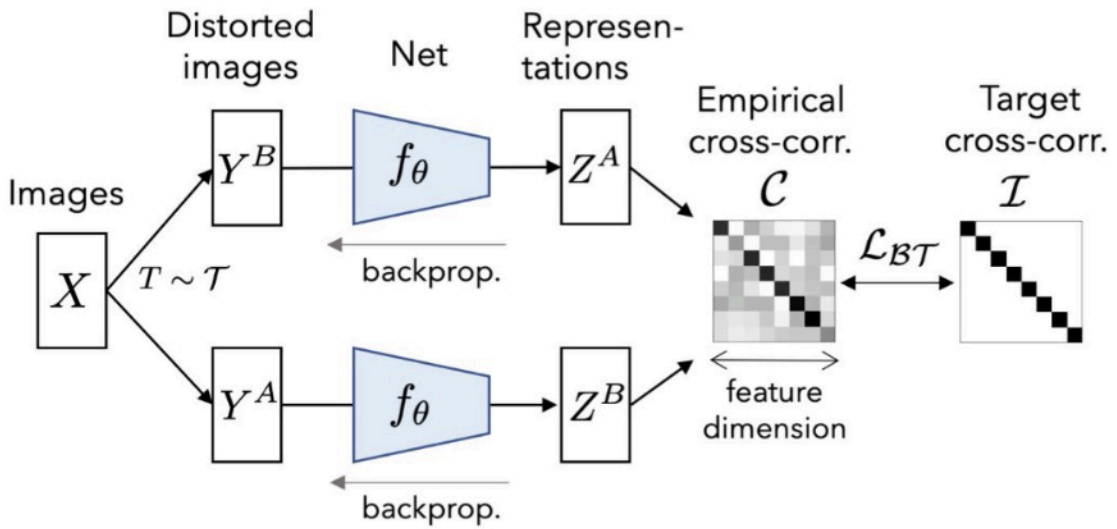
momentum update

FIFO update of the queue: the current minibatch becomes a new negative, the oldest minibatch leaves the queue

What if there are no negatives?

Barlow Twins (Zbontar et al. 2021)

- Learn to make the cross-correlation matrix between two output features for two distorted version of the same sample close to the identity.



$$\mathcal{L}_{\mathrm{BT}} = \underbrace{\sum_i (1 - \mathcal{C}_{ii})^2}_{\text{invariance term}} + \lambda \underbrace{\sum_i \sum_{i \neq j} \mathcal{C}_{ij}^2}_{\text{redundancy reduction term}}$$

$$\text{where } \mathcal{C}_{ij} = \frac{\sum_b \mathbf{z}^A_{b,i} \mathbf{z}^B_{b,j}}{\sqrt{\sum_b (\mathbf{z}^A_{b,i})^2} \sqrt{\sum_b (\mathbf{z}^B_{b,j})^2}}$$

(Zbontar et al. 2021)

slide by Weng & Kim, NeurIPS 2021 Tutorial

Concept Comparison